

1. INTRODUCCIÓN

Este proyecto consiste en el diseño, desarrollo e implementación de un portal web integral dedicado a la centralización, agrupación y gestión dinámica de un catálogo que alberga información detallada sobre distintas asociaciones médicas, fundaciones y entidades sin ánimo de lucro vinculadas al entorno de la salud y al hospital.

El presente documento técnico (dividido en siete capítulos principales más bibliografía) abarca todo el ciclo de vida del desarrollo del software: desde la viabilidad inicial, pasando por el análisis de requisitos, el diseño de la arquitectura (Frontend y Backend), hasta la implementación técnica, pruebas y conclusiones finales.

1.1. Objetivo y Alcance Previsto

La iniciativa surge de la necesidad de digitalizar los canales de colaboración impulsados por la Comisión de Participación Ciudadana del HUSC. Actualmente, el hospital difunde quincenalmente el trabajo de 26 asociaciones mediante campañas audiovisuales, pero carecía de un espacio centralizado interactivo donde pacientes y profesionales pudieran consultar este ecosistema de apoyo.

1.1.1. Objetivos Principales

El objetivo principal es la centralización del acceso a la información, facilitando al usuario (pacientes, familiares o profesionales sanitarios) el acceso estructurado a los datos de las asociaciones, tales como su misión, servicios, localización y métodos de contacto. Simultáneamente, se busca implementar una gestión autónoma, dotando a la administración del hospital de un panel de control privado para crear, modificar, eliminar y filtrar las asociaciones registradas sin depender de personal técnico. Finalmente, el proyecto se orienta a visibilizar la función que desarrollan estas agrupaciones en el voluntariado, el acompañamiento emocional y la educación comunitaria.

1.1.2. Alcance del Proyecto

El alcance de esta fase abarca la creación de una interfaz de usuario pública responsiva para consultar el directorio mediante filtros cruzados (categorías, etiquetas, ubicación y búsqueda de texto libre). A esto se suma el desarrollo de un panel de administración restringido por autenticación para la gestión del catálogo. En el backend, se implementa una API RESTful en Python para gestionar peticiones asíncronas, procesar imágenes y validar la integridad de los datos. La persistencia de datos se resuelve mediante almacenamiento en formato JSON, permitiendo salvaguardar el estado del catálogo sin requerir un motor de base de datos relacional en esta etapa inicial. El alcance excluye en esta fase la integración con el historial clínico de los pacientes del HUSC y la gestión de citas médicas a través del portal.

1.2. Estado del Arte

El portal web desarrollado es un directorio dinámico avanzado. Existen en la actualidad diversos organismos autonómicos y estatales que ofrecen servicios de catálogos similares en el ámbito de la salud. Para fundamentar el diseño de la solución, se ha realizado un análisis comparativo de portales de referencia:

Portal de Referencia	Descripción General	Características Clave	Mejora en el proyecto
Censo de Asociaciones en Salud (Junta de Andalucía)	Red de apoyo oficial que permite consultar las entidades legalmente inscritas.	Filtros por provincia, tipo de asociación y buscador de texto.	Nuestra propuesta añade una experiencia interactiva sin recarga de página al aplicar filtros.

Portal de Referencia	Descripción General	Características Clave	Mejora en el proyecto
Directorio de Pacientes Sacyl (Castilla y León)	Portal de salud que mantiene un registro público dividido por categorías clínicas.	Información de contacto operativo (teléfono, email, web, dirección física).	El catálogo implementa un modelo de etiquetado múltiple que complementa la clasificación jerárquica.
Directorio de Salud Mental (Andalucía)	Catálogo gestionado por la federación autonómica que lista asociaciones orientadas a la salud mental.	Estructura de datos (nombre, dirección, correo, presidencia).	El proyecto del HUSC incluye un flujo de solicitudes de inclusión desde el propio portal.

La solución asimila las prácticas estructurales del estado del arte, incorporando una arquitectura basada en el desacoplamiento entre cliente y servidor.

2. ESTUDIO DE VIABILIDAD

2.1. Introducción al Estudio de Viabilidad

En el presente capítulo se realiza un análisis de las condiciones técnicas, operativas y temporales que enmarcan el proyecto para evaluar su viabilidad.

El análisis evalúa la situación actual del entorno hospitalario y formula una propuesta de mejora. Posteriormente, se definen los perfiles de usuario, se delimitan las funcionalidades del sistema y se realiza un inventario de los recursos humanos, hardware y software necesarios. Finalmente, se establece la metodología de trabajo y la planificación temporal.

2.2. Objeto del Proyecto

2.2.1. Situación Actual

La iniciativa de participación ciudadana está impulsada por la Comisión de Participación Ciudadana del Hospital Universitario Clínico San Cecilio de Granada, la cual consolida la participación de 26 asociaciones y fundaciones.

El flujo de trabajo actual se basa en la difusión quincenal de campañas de concienciación mediante material audiovisual. Sin embargo, no se dispone de un repositorio digital unificado y público donde el ciudadano pueda buscar y filtrar este tejido asociativo de forma autónoma.

2.2.2. Propuesta de Mejora

Se propone el desarrollo e implementación de un Portal Web Centralizado. Esta herramienta permitirá a los ciudadanos consultar la información de las asociaciones vinculadas al hospital mediante un entorno filtrable. Asimismo, dotará a los administradores del hospital de un Panel de Control para añadir nuevas asociaciones, modificar datos existentes y revisar peticiones de ingreso, reduciendo la dependencia de soporte técnico externo para la actualización de contenidos.

2.2.3. Perfiles de Usuario del Proyecto

El sistema está diseñado en torno a un modelo de control de accesos basado en dos roles:

Perfil de Usuario	Nivel de Privilegios	Funcionalidades Principales
-------------------	----------------------	-----------------------------

Perfil de Usuario	Nivel de Privilegios	Funcionalidades Principales
Usuario No Registrado (Visitante)	Público (Nivel 1)	Acceso al directorio completo. Búsqueda por nombre/siglas. Uso de filtros de categoría, etiquetas y localización. Lectura de las fichas de detalle y envío de formularios de solicitud de alta.
Gestor de Contenidos (Administrador)	Privado (Nivel 2)	Acceso protegido mediante login. Capacidad para Crear, Leer, Actualizar y Eliminar (CRUD) asociaciones, categorías y etiquetas. Procesamiento de solicitudes de inclusión e importación masiva de datos (CSV).

2.2.4. Objetivos del Estudio

A corto plazo, el objetivo es facilitar el acceso público a los servicios de participación ciudadana. A medio plazo, se busca agilizar el trabajo administrativo mediante la centralización del directorio en formato digital. A largo plazo, se proyecta estructurar una API RESTful escalable que permita migrar el sistema de almacenamiento a un motor de base de datos relacional si fuera necesario.

2.3. Descripción del Sistema

2.3.1. Funcionalidades Troncales

La aplicación es una plataforma compuesta por una capa de presentación interactiva servida estáticamente, que se comunica con una API dinámica. Sus funcionalidades clave incluyen un motor de búsqueda y filtrado de múltiples variables. Se implementa un panel de control con formularios para la gestión de logotipos y la actualización de registros. Destaca la automatización del etiquetado de asociaciones. La seguridad de las operaciones administrativas se gestiona mediante autenticación por JSON Web Tokens (JWT), garantizando que las comunicaciones con el servidor de datos estén autenticadas.

2.4. Análisis de Recursos

2.4.1. Recursos Humanos

El ciclo de vida completo de la primera versión funcional recaerá sobre el alumnado de prácticas. Estos perfiles asumirán de manera coordinada el desarrollo del Frontend y del Backend durante su periodo formativo en la empresa.

2.4.2. Recursos Hardware

Entorno	Requisitos Mínimos	Justificación
Máquina Cliente	PC, Tablet o Smartphone con conexión a Internet (>56 Kbps)	El procesamiento reside en el servidor; el cliente únicamente renderiza la interfaz.
Servidor Físico / VPS	CPU de 1-2 núcleos, 1GB RAM, 10GB Almacenamiento HDD/SSD	El uso de JSON y FastAPI (Uvicorn) reduce la huella de memoria necesaria en comparación con stacks tradicionales.

2.4.3. Recursos Software

La pila tecnológica se ha seleccionado priorizando el rendimiento, la escalabilidad y el uso de software de código abierto.

Para el cliente, se requiere soporte en navegadores web modernos (Chromium, Gecko, WebKit). La interfaz se construye utilizando HTML5, CSS3 y Vanilla JavaScript, sin depender de frameworks de interfaz adicionales.

Para el servidor, se recomienda el despliegue sobre un sistema operativo basado en Linux, como Debian Server. El backend se programa en Python 3.9 o superior, ejecutando el framework FastAPI sobre el servidor Uvicorn.

El equipo se apoyará en Visual Studio Code como editor principal, Visual Paradigm para el modelado de diagramas UML, y Git para el control de versiones.

2.5. Planificación del Proyecto

2.5.1. Temporalización Inicial

El proyecto se enmarca entre el 15 de mayo y el 2 de junio de 2026, coincidiendo con el periodo de prácticas. Se estima una dedicación de 6 horas diarias en un marco de tiempo flexible adaptado a la operativa administrativa del hospital.

2.5.2. Metodología y Modelo de Desarrollo

Se ha optado por el Modelo Ágil Scrum en lugar de los modelos de desarrollo en cascada, debido a la necesidad de obtener retroalimentación continua por parte de la Comisión de Participación Ciudadana. El enfoque ágil facilita la entrega rápida de prototipos funcionales y la iteración sobre las interfaces conforme se reciben comentarios del equipo médico. La ejecución operativa se segmenta en sprints que abarcan la toma de requisitos, redacción de especificaciones, desarrollo del catálogo público, panel de administración y fase de pruebas.

2.6. Conclusiones de Viabilidad

Tras la evaluación de las dimensiones operativas, económicas y técnicas, se determina que el proyecto es viable. La elección de arquitecturas ligeras, como FastAPI con almacenamiento JSON, permite el cumplimiento de los plazos sin la necesidad de infraestructura de servidores complejos en la fase inicial. El producto final logrará digitalizar procesos previamente manuales y ofrecer una herramienta funcional para los pacientes y el personal hospitalario.

3. ANÁLISIS DEL PROYECTO

3.1. Introducción

En este capítulo se detallan los requerimientos funcionales y no funcionales que rigen la lógica de negocio y las restricciones del portal web del Hospital Universitario Clínico San Cecilio.

El análisis funcional define los perfiles de usuario, sus privilegios asociados y presenta diagramas de casos de uso y secuencia para ilustrar la interacción entre los actores y el sistema. Asimismo, se establecen las restricciones arquitectónicas y los objetivos de diseño requeridos para asegurar la mantenibilidad y seguridad de la plataforma.

3.2. Requerimientos Funcionales

Los requerimientos funcionales establecen las operaciones que debe realizar el sistema y los servicios ofrecidos a cada actor.

3.2.1. Visión General del Negocio

El proyecto tiene como núcleo funcional la centralización en un entorno web de todas las asociaciones vinculadas al hospital. El sistema debe permitir a los visitantes consultar un directorio interactivo, y proveer a los administradores de un panel de control para gestionar el catálogo público mediante la lectura y escritura de archivos JSON desde la API.

3.2.2. Requisitos Funcionales por Perfil de Usuario

El sistema aplica un control de acceso basado en roles. Para esta fase se definen dos niveles de privilegio activos, y se proyecta un tercero para futuras iteraciones:

3.2.2.1. Nivel 1: Usuario No Registrado (Visitante)

Representa al público en general. Sus funciones se limitan a la lectura y envío de solicitudes de contacto. Posee capacidad de búsqueda textual y filtrado por etiquetas, categorías médicas y ubicación geográfica. Tiene acceso a la ficha de detalles de cada asociación (descripción, servicios, redes sociales, vídeos). Asimismo, el visitante puede enviar propuestas de alta de nuevas asociaciones mediante un formulario web público. Finalmente, dispone de acceso al formulario de inicio de sesión para autenticarse si dispone de credenciales.

3.2.2.2. Nivel 2: Usuario Registrado (Gestor de Contenidos)

Representa al personal autorizado del hospital. Este perfil hereda las capacidades del visitante e incorpora privilegios de escritura. Para acceder a sus funciones privadas, debe iniciar sesión mediante credenciales y obtener un token JWT válido. Una vez autenticado, puede crear nuevas asociaciones en el catálogo, editar cualquier campo de asociaciones existentes y gestionar la subida de logotipos e imágenes de fondo globales. También dispone de permisos para eliminar registros permanentemente. El gestor tiene acceso a la bandeja de entrada para revisar y aprobar solicitudes de inclusión ciudadana. Además, puede utilizar una utilidad de importación masiva de datos y descargar copias de seguridad de todo el catálogo en formatos CSV y ODS.

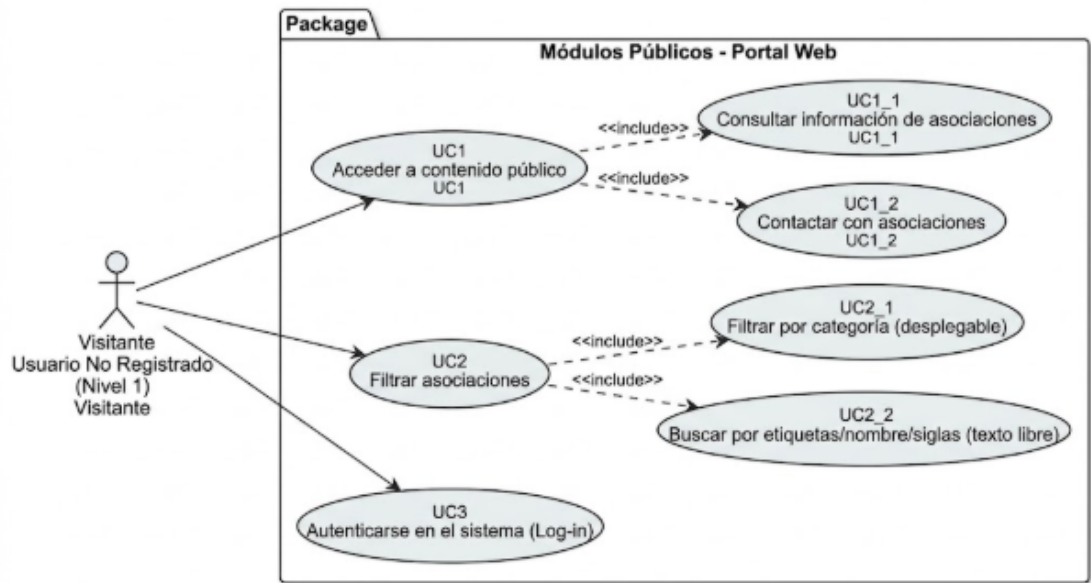
3.2.2.3. Nivel 3: Usuario Administrador (Escalabilidad Futura)

Este nivel se proyecta para fases posteriores de desarrollo. Estará encargado de la creación, gestión y auditoría de los usuarios gestores, asignando o revocando credenciales mediante una interfaz gráfica. En la fase actual, las credenciales del Nivel 2 están gestionadas internamente en el backend.

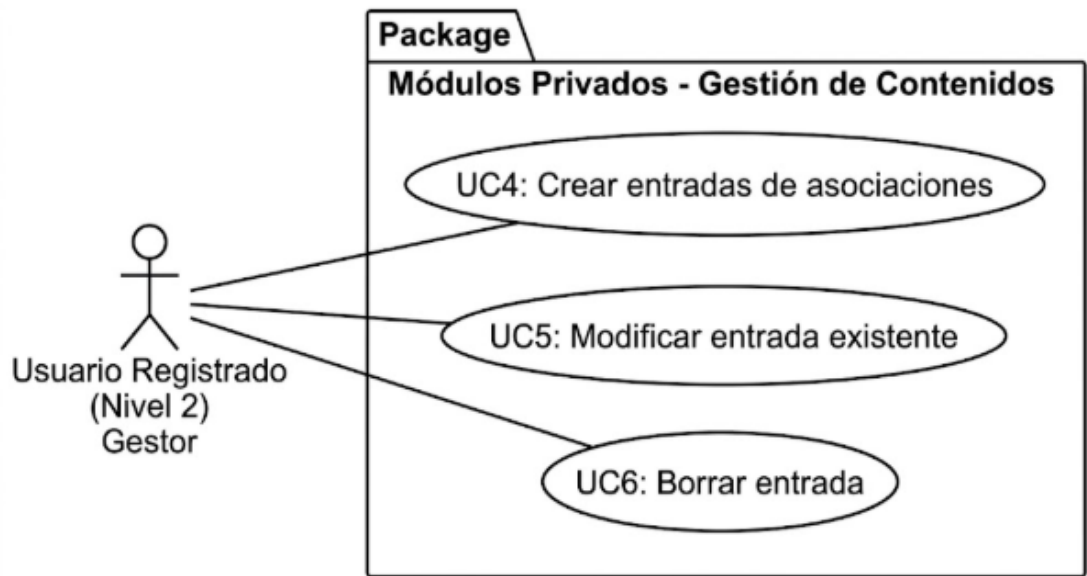
3.2.3. Diagramas de Casos de Uso

A continuación se modela de forma gráfica la relación entre los actores y los casos de uso principales.

3.2.3.1. Actores No Registrados (Visitantes)



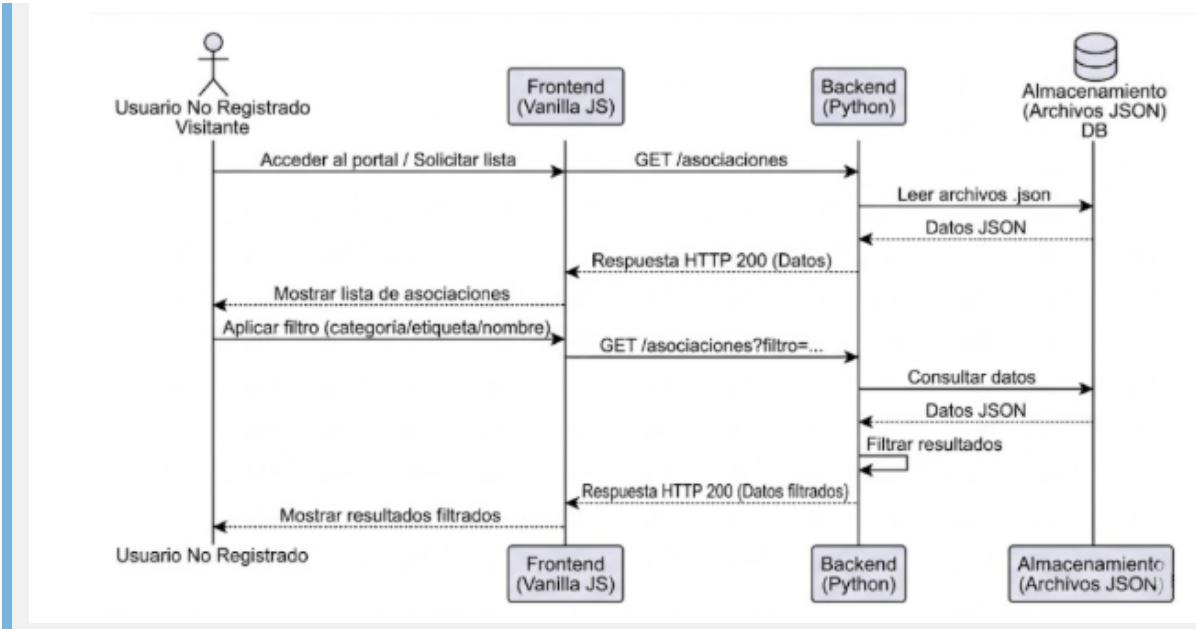
3.2.3.2. Actores Registrados (Gestores)



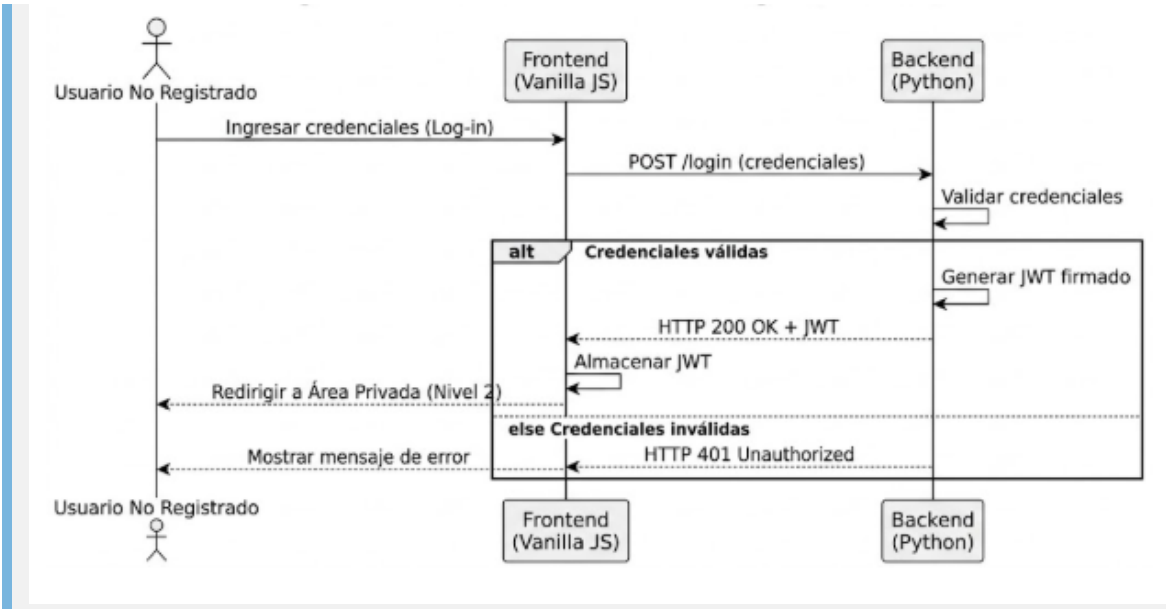
3.2.4. Diagramas de Secuencia

Estos diagramas detallan el flujo de mensajes asíncronos entre el cliente, la API y la capa de almacenamiento de datos.

3.2.4.1. Flujo de Lectura y Autenticación (Visitantes)



3.2.4.2. Flujo de Escritura y Edición (Gestores de Contenido)



3.2.5. Menús de Navegación Condicionales

La interfaz muestra los elementos del menú principal en función de la sesión activa del usuario. Durante la navegación pública, el menú incluye enlaces al inicio, el catálogo y el formulario de contacto. Si se detecta una sesión administrativa válida, la interfaz muestra los enlaces adicionales para la gestión de entidades y transforma el enlace de acceso principal en una opción de cierre de sesión.

3.3. Requerimientos No Funcionales

Los requerimientos no funcionales definen las restricciones tecnológicas, estándares de rendimiento y normativas de seguridad de la aplicación.

3.3.1. Restricciones de Arquitectura y Seguridad

La arquitectura del sistema requiere la separación de capas mediante una API RESTful. El frontend no debe realizar operaciones directas sobre el sistema de archivos de datos; toda lectura o escritura debe realizarse exclusivamente

mediante peticiones a los endpoints del servidor en Python.

En cuanto al control de accesos, el servidor opera bajo un modelo sin estado (stateless). La validación de privilegios se realiza verificando el token criptográfico JWT adjunto en la cabecera de las peticiones HTTP. Si el token está ausente, manipulado o expirado, el servidor debe devolver un error de autorización 401.

Para prevenir la inyección de código (Cross-Site Scripting), el frontend debe utilizar propiedades de renderizado de texto plano (`textContent`) en lugar de insertar HTML dinámicamente cuando renderice datos externos. Además, la API backend implementa validaciones estrictas usando Pydantic, garantizando que el servidor compruebe los tipos y campos obligatorios antes de escribir en disco, con el fin de evitar corrupciones en los archivos JSON.

3.3.2. Objetivos de Diseño y Experiencia de Usuario (UX)

La plataforma debe implementar un diseño visual centralizado basado en variables CSS, permitiendo futuras actualizaciones a la paleta de colores corporativa del hospital.

La interfaz prioriza la claridad y legibilidad, aplicando un diseño minimalista y sobrio que utiliza predominantemente colores institucionales sobre fondos de alto contraste.

La accesibilidad y adaptabilidad a dispositivos (Responsive Design) es obligatoria en todas las vistas. El catálogo, las ventanas modales y el panel de administración deben reestructurarse dinámicamente mediante propiedades como CSS Grid y Flexbox, asegurando la usabilidad en monitores de escritorio, tablets y dispositivos móviles sin generar barras de desplazamiento horizontales.

4. DISEÑO DE LA APLICACIÓN

En este capítulo se abordan las decisiones tecnológicas y metodológicas que fundamentan la arquitectura del proyecto. Se detallan los entornos de desarrollo elegidos, la configuración del servidor, la distribución de capas y la estructura de la base de datos, así como las medidas de seguridad implementadas.

4.1. Selección del Entorno de Desarrollo

Para la construcción del Catálogo de Asociaciones del Hospital Universitario Clínico San Cecilio, se ha optado por un conjunto de herramientas modernas, eficientes y de código abierto, con el objetivo de desarrollar una plataforma robusta y mantenible.

4.1.1. Lado del Cliente (Frontend)

El entorno de usuario, encargado de la presentación visual y la interacción, se ha desarrollado utilizando los estándares web nativos. Se ha decidido prescindir del uso de frameworks pesados de JavaScript para reducir el peso global de la aplicación y acelerar los tiempos de carga iniciales.

Para la estructura semántica de la información se emplea HTML5, asegurando la accesibilidad y el posicionamiento orgánico. La presentación y la maquetación visual se controlan a través de hojas de estilo en cascada (CSS3), utilizando módulos de diseño responsivo como Flexbox y CSS Grid. Toda la interactividad del portal, incluyendo el filtrado del catálogo, las peticiones asíncronas al servidor y la manipulación del Document Object Model (DOM), se gestiona mediante Vanilla JavaScript.

4.1.2. Lado del Servidor (Backend)

El motor de la aplicación, responsable de procesar la lógica de negocio y controlar el acceso a los datos, se ha desarrollado en Python. La elección de este lenguaje se basa en su sintaxis clara y su extensa adopción en el desarrollo backend.

En este entorno, se ha integrado el framework web FastAPI, reconocido por su rendimiento, naturaleza asíncrona y vinculación nativa con el sistema de validación Pydantic. Para el despliegue de esta arquitectura, se emplea el servidor ASGI Uvicorn, el cual procesa múltiples peticiones concurrentes de manera eficiente con un bajo consumo de recursos computacionales. Esta combinación proporciona tiempos de respuesta óptimos para las operaciones sobre el directorio de datos.

4.1.3. Herramientas de Apoyo al Desarrollo

El proceso de desarrollo, depuración y control de versiones se ha realizado utilizando el Entorno de Desarrollo Integrado Visual Studio Code. Esta herramienta ofrece un amplio ecosistema de extensiones y una terminal integrada, facilitando el trabajo unificado entre el cliente y el servidor.

El diseño preliminar de las interfaces y la concepción analítica de los flujos de interacción previos a la codificación se apoyaron en herramientas gráficas de modelado, asegurando que la arquitectura respondiera a las necesidades funcionales establecidas durante el estudio de viabilidad.

4.2. CONFIGURACIÓN DE LA PLATAFORMA

Tras analizar los entornos de desarrollo, es necesario concretar la configuración técnica de la plataforma que dará servicio a la aplicación en producción.

La aplicación web debe ser hospedada en un servidor ASGI de alto rendimiento, como Uvicorn, corriendo sobre el framework FastAPI. El servidor debe configurarse para soportar protocolos seguros (HTTPS con certificados SSL/TLS) y estar accesible de manera remota. Se recomienda realizar el despliegue sobre un sistema operativo basado en Linux, como Debian Server, debido a que ofrece estabilidad probada y una optimización eficiente de los recursos del sistema frente a cargas de trabajo asíncronas.

El servidor requiere soporte para Python 3.9 o superior y tener instaladas las librerías declaradas en el archivo `requirements.txt`. Entre estas dependencias destacan PyJWT, para la firma y validación de tokens JSON Web Token (JWT); Pydantic, para la validación estructural de esquemas de datos; y python-multipart, para el procesamiento de peticiones en formato URL-encoded durante la autenticación.

Adicionalmente, el servidor debe disponer de permisos de lectura y escritura sobre el sistema de archivos local. Esto es necesario para operar con los archivos estructurados en formato JSON (`asociaciones.json`, `categorias.json`, `etiquetas.json`, `solicitudes.json` y `config.json`), para el almacenamiento de los archivos de imagen subidos al directorio `html/img/logos/`, y para la lectura en memoria requerida durante la importación masiva de datos estructurados en formato CSV.

4.3. CAPAS DE LA APLICACIÓN

La arquitectura lógica de la aplicación se estructura en tres capas tecnológicas bien diferenciadas, garantizando un código modular, escalable y fácil de mantener.

4.3.1. Entorno de usuario

El entorno de usuario, o interfaz cliente, es el área visual de interacción y presentación de datos. Su construcción se basa en la combinación de HTML, CSS y JavaScript, organizando la información de manera semántica y accesible.

El diseño visual está gobernado por una hoja de estilos centralizada que utiliza variables para definir los colores corporativos y las tipografías del hospital. Se emplea CSS Grid y Flexbox para organizar los componentes en un diseño responsivo que se adapta correctamente a pantallas de escritorio, tablets y dispositivos móviles. Se han definido estilos específicos para la cabecera, el pie de página, las ventanas modales y los formularios, manteniendo una coherencia visual en toda la aplicación.

La estructura general de las páginas se compone de una cabecera fija con el logotipo institucional y el menú de navegación, un contenedor central destinado a albergar el contenido específico de cada vista (como la cuadrícula de asociaciones o los formularios), y un pie de página que contiene información legal y de contacto. El menú de navegación se adapta de forma dinámica, mostrando opciones administrativas únicamente si detecta una sesión válida.

En el catálogo principal, la información se presenta mediante un sistema de tarjetas interactivas dispuestas en cuadrícula. Estas tarjetas muestran los datos básicos de cada asociación y, al interactuar con ellas, desencadenan la apertura de una ventana modal superpuesta que exhibe la ficha completa de la entidad. Esta ventana modal también sirve como interfaz de edición para el administrador, incluyendo botones para modificar o eliminar el registro visualizado.

A nivel estructural, la aplicación sigue un esquema semántico HTML similar al siguiente pseudocódigo:

```
INICIO HTML5
HEAD
  METADATOS (charset, viewport)
  VÍNCULO CSS (style_Form.css)
FIN HEAD
BODY
  HEADER
    IMG (Logotipo de la empresa)
    NAV
      UL (Menú de navegación: Inicio, Asociaciones, Panel)
    FIN NAV
  FIN HEADER
  MAIN (Contenedor principal centrado, max-width: 900px)
    SECTION id="autenticacion"
      DIV class="login-container"
        FORM
          INPUT type="text" (Usuario)
          INPUT type="password" (Contraseña)
          BUTTON type="submit" (Entrar)
        FIN FORM
      DIV class="error-msg" (Mensaje de error interactivo)
    FIN DIV
  FIN SECTION
  SECTION id="contenido-galeria"
    DIV class="container-tarjetas" (Display: grid)
      ARTICLE class="tarjeta"
        IMG (Logo asociación)
        H3 (Nombre asociación)
        DIV class="insignia-filtro" (Siglas)
        DIV (Contenedor de etiquetas múltiples)
        BUTTON (Ver Detalles / Modal)
      FIN ARTICLE
    FIN DIV
  FIN SECTION
  DIV id="modal-detalle" class="modal-activo/modal-oculto" (Overlay)
    DIV class="modal-contenido"
      BUTTON (Cerrar modal - ×)
      DIV class="modal-cabecera"
        IMG (Logo asociación)
        H2 (Nombre asociación)
        H4 (Siglas)
      FIN DIV
    FIN DIV
```

```

        P (Descripción y categorías)
        DIV (Contactos de la asociación)
        DIV class="caja-texto" (Cartera de servicios)
        DIV id="contenedor-borrado" (Acciones de administrador)
            BUTTON (Modificar)
            BUTTON (Eliminar)
        FIN DIV
    FIN DIV
FIN DIV
FIN MAIN
FOOTER
    DIV class="footer-contenido" (Columnas grid)
        DIV (Columna 1: Datos corporativos)
        DIV (Columna 2: Privacidad)
        DIV (Columna 3: Contacto)
    FIN DIV
    DIV class="footer-bottom"
        P (Copyright)
    FIN DIV
FIN FOOTER
VÍNCULO JS
FIN BODY
FIN HTML5

```

4.3.2. Motor de la aplicación

El motor de la aplicación actúa como intermediario lógico entre el usuario y la capa de datos. Para organizar esta interacción, se ha adaptado el patrón de arquitectura Modelo-Vista-Controlador (MVC) a un entorno web asíncrono y desacoplado.

La capa de la Vista está representada por los archivos estáticos del frontend (HTML, CSS y JS), responsables de capturar los eventos del usuario y mostrar la información solicitada. El rol de Controlador lo asume el servidor backend desarrollado en Python con FastAPI. Su función principal es exponer rutas RESTful, recibir las peticiones HTTP asíncronas, validar las sesiones mediante tokens JWT y gestionar el enrutamiento hacia la lógica correspondiente. El Modelo queda definido por las clases validadoras de Pydantic, que establecen los esquemas estrictos de datos y supervisan la integridad de la información antes de cualquier interacción con los archivos físicos.

El servidor FastAPI, al ejecutarse sobre el servidor ASGI Uvicorn, procesa altos volúmenes de peticiones concurrentes de manera eficiente. Entre las funcionalidades implementadas en esta capa destacan la manipulación individual de registros mediante operaciones RESTful, la validación estricta de esquemas de datos, la gestión de la subida de recursos multimedia con renombrado seguro (incluyendo fondos de pantalla dinámicos), y un módulo bidireccional para la importación masiva y exportación de copias de seguridad de datos estructurados en formatos CSV y ODS (OpenDocument Spreadsheet). Durante el desarrollo, se utilizaron herramientas auxiliares como la interfaz gráfica autogenerada de Swagger (OpenAPI) para probar los endpoints, así como las herramientas de desarrollo del navegador para monitorizar el tráfico de red y las peticiones asíncronas.

4.3.3. Capa de datos

La capa de datos constituye el repositorio físico donde se almacena la información del proyecto. La persistencia se basa en un formato de texto estructurado mediante archivos JSON, alojando registros de asociaciones, categorías, etiquetas y configuraciones.

Las operaciones de lectura y escritura sobre estos archivos son gestionadas exclusivamente por el backend de FastAPI. Este acceso está centralizado en funciones específicas de entrada/salida que incluyen manejo de excepciones para prevenir corrupciones en caso de errores durante la manipulación de los datos. La información procesada por el

servidor se serializa y se guarda en disco manteniendo un formato indentado, lo que facilita tanto su legibilidad directa por parte del equipo de desarrollo como su consistencia estructural a lo largo del ciclo de vida de la aplicación.

4.4. ESTRUCTURA DE LA BASE DE DATOS

La persistencia de datos del catálogo se implementa mediante un sistema de almacenamiento basado en archivos de texto plano en formato JSON. Esta arquitectura elimina la necesidad de motores de bases de datos relacionales externos, proporcionando portabilidad al proyecto y simplificando el proceso de copias de seguridad mediante la duplicación del directorio de datos.

El repositorio de datos se divide lógicamente en cuatro archivos independientes alojados en el directorio de almacenamiento del servidor. Cada archivo actúa como una colección de documentos, relacionándose entre sí mediante identificadores únicos.

4.4.1. Archivo Principal: asociaciones.json

El archivo `asociaciones.json` contiene la matriz principal del catálogo público. Cada objeto almacenado en este archivo representa una asociación médica y se estructura en bloques descriptivos definidos.

El bloque de identidad incluye un identificador único asignado automáticamente por el sistema. Además, contiene el nombre oficial de la asociación, sus siglas y la ruta estática o URL del logotipo representativo.

El bloque descriptivo almacena un campo de texto con la descripción detallada de la entidad y otro para la cartera de servicios ofrecidos. Adicionalmente, incluye el identificador numérico de la categoría a la que pertenece y un arreglo de identificadores correspondientes a sus etiquetas asociadas, fundamentales para los procesos de filtrado.

El bloque de contacto y geolocalización abarca los datos de comunicación directa, estructurados en un arreglo de contactos (incluyendo teléfono, correo electrónico y sitio web). Además, incluye un sub-objeto de ubicación con la dirección postal (país, comunidad, provincia, municipio) y una lista de identificadores de vídeos para la galería multimedia.

4.4.2. Taxonomía y Ordenación: categorias.json y etiquetas.json

El sistema emplea dos archivos complementarios, `categorias.json` y `etiquetas.json`, para normalizar la clasificación de las asociaciones, evitando redundancias en los textos descriptivos.

El archivo de categorías contiene un arreglo de objetos compuestos por un identificador único y el nombre de la categoría principal (por ejemplo, Oncología o Trastornos Neurológicos). Las asociaciones referencian este identificador en su campo correspondiente.

El archivo de etiquetas sigue la misma estructura, almacenando identificadores numéricos y cadenas de texto para características secundarias o servicios específicos (como Voluntariado o Apoyo Psicológico). Esta separación relacional permite actualizar el nombre de una categoría o etiqueta una sola vez y que el cambio se refleje en todas las asociaciones vinculadas.

4.4.3. Bandeja de Entrada Segura: solicitudes.json

El archivo `solicitudes.json` se encarga de almacenar las peticiones de inclusión enviadas por los ciudadanos mediante el formulario público.

La estructura interna de este archivo combina la información descriptiva de la asociación solicitada con metadatos de auditoría requeridos para su gestión. Cada objeto incluye una marca temporal de creación, un identificador provisional, y un campo de estado que por defecto se define como "pendiente". Esta separación física entre el catálogo oficial y la bandeja de entrada asegura que los datos sin validar no comprometan el archivo principal de asociaciones hasta que no sean revisados y aprobados explícitamente por el administrador.

4.5. ARQUITECTURA DE LA APLICACIÓN

La arquitectura de la aplicación se fundamenta en la separación de responsabilidades entre el frontend, el backend y el sistema de almacenamiento. El diseño adopta un enfoque inspirado en el patrón Modelo-Vista-Controlador (MVC), adaptado a un entorno asíncrono donde el cliente web y el servidor operan de manera independiente comunicándose mediante una API RESTful.

El flujo general de interacción comienza cuando el usuario realiza una acción en el navegador, como buscar una asociación o enviar un formulario. El código JavaScript (Vista) intercepta el evento, recolecta los datos necesarios y genera una petición HTTP asíncrona (Fetch) hacia el servidor. Si la petición requiere privilegios administrativos, el cliente adjunta automáticamente el token JWT en la cabecera de la solicitud.

El servidor FastAPI (Controlador) recibe la petición y, si corresponde a una ruta protegida, verifica la validez de la firma criptográfica y la fecha de expiración del token. Si la autenticación falla, se devuelve un error de acceso no autorizado. A continuación, el servidor valida la estructura y los tipos de los datos recibidos utilizando los esquemas de Pydantic (Modelo). Una vez superada esta validación, se ejecutan las operaciones correspondientes de lectura o escritura en los archivos JSON. Finalmente, el servidor responde al cliente con los datos actualizados, y el JavaScript se encarga de modificar el Modelo de Objetos del Documento (DOM) para reflejar los cambios en la interfaz sin necesidad de recargar la página.

4.5.1. Flujos de Acción en la Arquitectura

Las interacciones dentro del portal web pueden clasificarse en operaciones pasivas de lectura, operaciones de administración con privilegios y flujos de recolección de solicitudes externas.

Las operaciones pasivas de lectura corresponden a las búsquedas y filtrados en el catálogo público. Al seleccionar una categoría o escribir en el buscador, el cliente realiza peticiones asíncronas para obtener la lista actualizada de asociaciones. El servidor procesa la solicitud, lee la información correspondiente de los archivos JSON y devuelve los datos estructurados. El frontend renderiza inmediatamente los resultados en la cuadrícula de la galería, proporcionando una experiencia de usuario ágil y sin recargas de la interfaz.

Las operaciones de gestión administrativa implican tareas de escritura, actualización o eliminación de registros, y requieren autenticación explícita. El administrador, desde su panel de control, puede modificar los datos de una asociación, subir nuevos logotipos o gestionar categorías. En el caso de recursos multimedia, el servidor recibe el archivo, valida su extensión y tamaño, lo renombra con un UUID para evitar colisiones y lo almacena en el directorio estático. Simultáneamente, los datos descriptivos se validan mediante Pydantic. Una vez que ambas validaciones son exitosas, la base de datos JSON se actualiza y la interfaz del administrador muestra una notificación de éxito, actualizando la vista del catálogo.

Por último, el flujo de recolección de solicitudes ciudadanas combina elementos de acceso público con gestión privada. Usuarios anónimos pueden enviar propuestas de nuevas asociaciones mediante un formulario web. Este formulario incorpora validaciones en tiempo real y medidas antispam pasivas (como campos ocultos Honeypot) que bloquean envíos automatizados desde el cliente. Las solicitudes legítimas son recibidas por el servidor, validadas y almacenadas en un archivo JSON independiente bajo el estado "pendiente". Estas peticiones permanecen aisladas del catálogo principal hasta que un administrador autenticado revisa el listado desde su panel. Tras evaluar los datos, el administrador puede aprobar la solicitud, lo que transfiere la información al archivo principal de asociaciones, o rechazarla, eliminando el registro de la bandeja de entrada.

4.6. INTERFAZ (ENTORNOS Y VISTAS)

La plataforma web desarrollada para el Hospital Universitario se organiza en un conjunto de vistas interactivas, diseñadas para resolver necesidades funcionales específicas según el rol del usuario que accede al sistema.

4.6.1. Entorno Público: Catálogo General y Galería

La vista pública principal es el catálogo de asociaciones médicas. Este espacio presenta un diseño limpio y minimalista, priorizando la visualización de la información mediante una cuadrícula de tarjetas modulares adaptativas.

En la zona superior, el usuario dispone de herramientas de filtrado reactivo. El campo de búsqueda de texto opera de forma predictiva, actualizando los resultados con cada pulsación sin necesidad de un botón de confirmación. Adicionalmente, se incluyen selectores desplegables dinámicos que permiten segmentar la cuadrícula seleccionando una categoría médica general o aplicando un cruce de etiquetas secundarias para afinar la búsqueda.

Cada tarjeta de la galería muestra el logotipo de la asociación, su denominación, las siglas y las etiquetas correspondientes. Para salvaguardar la armonía del diseño ante datos extremos, la interfaz aplica medidas de protección visual avanzadas:

- **Truncado de Nombres:** Los títulos de las asociaciones con una longitud superior a 55 caracteres son truncados e incorporan puntos suspensivos (. . .), junto con un atributo `title` nativo que desvela el nombre completo al pasar el cursor (hover).
- **Control de Desbordamiento en Etiquetas:** El contenedor de tags tiene delimitada una altura máxima de 3.6rem y envoltura limpia (`overflow: hidden`), impidiendo que un exceso de etiquetas desplace la maquetación de la tarjeta.

Al seleccionar una tarjeta, la plataforma abre una ventana modal superpuesta que oscurece el fondo para centrar la atención. Esta vista detallada presenta la descripción completa de la entidad, su cartera de servicios, información de contacto, geolocalización mediante mapas y enlaces a dominios oficiales o redes sociales.

4.6.2. La Pasarela del Solicitante Anónimo

El entorno para el solicitante anónimo es un formulario diseñado para gestionar la recepción de nuevas solicitudes de inclusión en el catálogo.

Esta vista adopta una disposición centralizada para facilitar la introducción de datos de forma secuencial. El formulario requiere que la asociación preocupe su información de identidad (nombre, siglas, logotipo), su clasificación mediante el selector de categorías y etiquetas, y sus datos de contacto (teléfono, web, ubicación). El sistema incluye un mecanismo de validación instantánea en el cliente que resalta en color rojo los campos que no cumplen con los formatos requeridos o que han sido omitidos, impidiendo el envío de la solicitud hasta que todos los datos sean válidos. Asimismo, se requiere la aceptación de los términos de protección de datos.

4.6.3. Identificación y Control de Acceso

El acceso al panel de administración está protegido por una vista de autenticación dedicada.

En este entorno, el sistema solicita al usuario sus credenciales: nombre de usuario y contraseña ofuscada. Los datos introducidos se envían de forma asíncrona al servidor backend, donde se validan contra las variables de entorno seguras. Si la validación falla por credenciales incorrectas, la interfaz muestra un mensaje de error claro, impidiendo el acceso. Si las credenciales son válidas, el servidor devuelve un token JWT y la vista redirige automáticamente al usuario al panel de control administrativo.

4.6.4. Panel de Administración

El panel de administración es el entorno privado desde el cual se gestionan los contenidos del catálogo general, las solicitudes pendientes y la taxonomía del portal.

El primer módulo de este panel es la bandeja de solicitudes de inclusión. Aquí se listan en formato tabular las peticiones recibidas desde el formulario público. El administrador puede revisar el contenido detallado de cada solicitud y dispone de controles para aprobarla, transfiriendo los datos al catálogo oficial de asociaciones, o rechazarla, eliminando la solicitud del sistema.

Desde la vista de galería, el administrador cuenta con botones integrados en cada tarjeta para modificar o eliminar entradas existentes. Al seleccionar la opción de modificación o creación de registros, se despliega un formulario avanzado estructurado bajo una **secuencia ergonómica y lógica de lectura**:

1. **Identificación y Logo:** Nombre, Siglas y previsualizador del logotipo del centro.
2. **Taxonomía:** Selectores dinámicos de Categorías y Etiquetas múltiples.
3. **Localización:** Datos de ubicación física detallados.
4. **Contenido Activo:** Descripción detallada del organismo, seguida inmediatamente por la Cartera de Servicios.
5. **Vías de Comunicación:** Campos de contacto del personal y enlaces web.
6. **Videos de YouTube:** Galería de soporte multimedia pegada al final de la interacción.

4.6.4.1. Módulo de Gestión de Categorías

Ubicado de forma independiente debajo del formulario principal, este módulo proporciona una cuadrícula interactiva con todas las categorías activas.

- **Transparencia de Datos:** Cada tarjeta de categoría muestra el número exacto de asociaciones clínicas vinculadas.
- **Borrado Inteligente:** Las categorías con cero registros pueden ser eliminadas permanentemente tras confirmación (botón ✕). Las categorías activas muestran el botón deshabilitado e incorporan un tooltip en hover que indica al administrador las asociaciones asignadas que debe liberar previamente, manteniendo la integridad referencial.
- **Sincronización en Caliente:** Las acciones sobre este panel actualizan instantáneamente los menús desplegables del formulario sin necesidad de recargar la página.

4.7. SEGURIDAD Y CONTROL DE ACCESO

Debido a que la aplicación web gestiona información sensible de asociaciones de salud ligadas al hospital y ofrece un panel de administración para manipular los datos, la seguridad es un pilar fundamental en el desarrollo del sistema. A continuación se detallan las medidas de protección implementadas tanto en el servidor como en el cliente para garantizar la confidencialidad, integridad y disponibilidad del portal.

4.7.1. Autenticación y Autorización basada en JWT (JSON Web Tokens)

Para la gestión de sesiones de administración se ha implementado un esquema de autenticación desacoplado y sin estado (*stateless*) utilizando **JSON Web Tokens (JWT)**. Este mecanismo evita la necesidad de almacenar sesiones en el servidor, simplificando la arquitectura y reduciendo la superficie de ataque. A continuación se describe en profundidad cada componente del sistema.

4.7.1.1. Configuración de Seguridad en el Servidor

El módulo `server.py` define tres constantes críticas de seguridad que gobiernan todo el ciclo de vida de la autenticación:

Constante	Origen	Valor por Defecto	Propósito
SECRET_KEY	Variable de entorno SECRET_KEY	Cadena de desarrollo (se debe cambiar en producción)	Clave criptográfica utilizada para firmar y verificar la integridad de los tokens JWT.

Constante	Origen	Valor por Defecto	Propósito
ADMIN_USERNAME	Variable de entorno ADMIN_USERNAME	admin	Nombre de usuario del administrador contra el que se validan las credenciales.
ADMIN_PASSWORD	Variable de entorno ADMIN_PASSWORD	1234	Contraseña del administrador. Se externaliza mediante variables de entorno para no incrustar secretos en el código fuente.
ALGORITHM	Código fuente (constante fija)	HS256	Algoritmo de hash simétrico HMAC-SHA256 utilizado para la firma criptográfica del token.

```
# Configuración de Seguridad (server.py, líneas 20-23)
SECRET_KEY = os.environ.get("SECRET_KEY",
"cambia_esto_por_una_clave_larga_y_secreta_en_produccion")
ADMIN_USERNAME = os.environ.get("ADMIN_USERNAME", "admin")
ADMIN_PASSWORD = os.environ.get("ADMIN_PASSWORD", "1234")
ALGORITHM = "HS256"
```

Adicionalmente, se declara un esquema de autenticación OAuth2 de tipo *Password Bearer* que instruye a FastAPI sobre dónde buscar el token en las peticiones entrantes:

```
# Esquema OAuth2 (server.py, línea 48)
oauth2_scheme = OAuth2PasswordBearer(tokenUrl="token")
```

Este esquema espera que el token se transmita en la cabecera HTTP *Authorization* con el prefijo *Bearer*, siguiendo el estándar RFC 6750.

4.7.1.2. Flujo Completo de Autenticación

El ciclo de autenticación sigue un flujo de 4 fases bien definidas:

Fase 1 — Envío de Credenciales (Cliente → Servidor)

El administrador introduce su usuario y contraseña en el formulario de login (*login.html*). El script *login.js* serializa las credenciales en formato *application/x-www-form-urlencoded* (requerido por el estándar OAuth2 Password Flow) y las envía mediante una petición *POST* asíncrona al endpoint */token*:

```
// login.js – Envío de credenciales al servidor
const formData = new URLSearchParams();
formData.append('username', usuario);
formData.append('password', password);

const response = await fetch('/token', {
  method: 'POST',
  headers: { 'Content-Type': 'application/x-www-form-urlencoded' },
  body: formData
});
```


Fase 2 — Validación de Identidad y Generación del Token (Servidor)

El endpoint `POST /token` recibe las credenciales a través del objeto `OAuth2PasswordRequestForm` inyectado automáticamente por FastAPI. El servidor compara las credenciales recibidas contra las variables de entorno seguras:

```
# server.py – Endpoint de autenticación (líneas 145-155)
@app.post("/token")
async def login(form_data: OAuth2PasswordRequestForm = Depends()):
    if form_data.username == ADMIN_USERNAME and form_data.password ==
ADMIN_PASSWORD:
        access_token = create_access_token(data={"sub": form_data.username})
        return {"access_token": access_token, "token_type": "bearer"}
    raise HTTPException(
        status_code=status.HTTP_401_UNAUTHORIZED,
        detail="Credenciales incorrectas",
        headers={"WWW-Authenticate": "Bearer"},
    )
```

Si las credenciales son válidas, se invoca la función `create_access_token()`, que construye el *payload* del token con dos campos (*claims*) fundamentales. El primero de ellos es el identificador del sujeto o `sub` (subject), el cual identifica inequívocamente al usuario autenticado, en este caso, el nombre de usuario del administrador. El segundo campo es la expiración o `exp` (expiration), que actúa como una marca temporal UTC definiendo el momento exacto e inamovible en que el token caducará y dejará de ser válido, configurado estrictamente a 30 minutos desde el instante de su creación.

```
# server.py – Generación del token firmado (líneas 127-132)
def create_access_token(data: dict):
    to_encode = data.copy()
    expire = datetime.utcnow() + timedelta(minutes=30)
    to_encode.update({"exp": expire})
    return jwt.encode(to_encode, SECRET_KEY, algorithm=ALGORITHM)
```

La función `jwt.encode()` de la librería PyJWT firma criptográficamente el payload completo `{"sub": "admin", "exp": <timestamp>}` utilizando la `SECRET_KEY` y el algoritmo `HS256`, produciendo una cadena codificada en Base64URL compuesta por tres segmentos separados por puntos: `header.payload.signature`.

Fase 3 — Almacenamiento del Token (Cliente)

Cuando el servidor responde con el token, el cliente lo almacena de forma persistente en el `localStorage` del navegador bajo la clave `adminToken`, y redirige al panel de administración:

```
// login.js – Almacenamiento post-autenticación
const data = await response.json();
localStorage.setItem('adminToken', data.access_token);
window.location.href = 'index.html';
```

Fase 4 — Inyección Automática en Peticiones Protegidas (Cliente → Servidor)

A partir de este momento, todas las operaciones de escritura, modificación o eliminación que realice el administrador requieren la inclusión del token en la cabecera HTTP. El script `crearEntrada.js` centraliza esta lógica en una

función auxiliar `apiRequest()` que inyecta automáticamente la cabecera `Authorization: Bearer <token>` en cada petición:

```
// crearEntrada.js – Inyección centralizada del token (líneas 43-88)
async function apiRequest(url, method = 'GET', body = null) {
  const headers = {
    'Authorization': `Bearer ${token}`
  };
  if (body) {
    headers['Content-Type'] = 'application/json';
  }
  const response = await fetch(url, { method, headers, body: body ?
JSON.stringify(body) : null });

  // Detección proactiva de expiración de sesión
  if (response.status === 401) {
    alert("Tu sesión ha expirado.");
    localStorage.removeItem('adminToken');
    window.location.href = 'login.html';
    return null;
  }
  // ...
}
```

4.7.1.3. Verificación y Validación del Token en el Servidor

Cada endpoint protegido del servidor inyecta como dependencia la función `verify_token()`, que realiza una rigurosa triple validación del token recibido. En primer lugar, ejecuta la verificación de la firma criptográfica mediante `jwt.decode()`, que recalcula la firma HMAC-SHA256 del header y payload recibidos utilizando la `SECRET_KEY` del servidor; si la firma no coincide, indicando manipulación maliciosa del token, lanza la excepción `jwt.InvalidTokenError`. En segundo lugar, procede a la verificación de vigencia temporal, donde la librería comprueba automáticamente el claim `exp`; si el instante actual supera la fecha de expiración, el acceso es denegado lanzando `jwt.ExpiredSignatureError`. Por último, se ejecuta la verificación de la identidad del sujeto, comprobando, una vez decodificado, que el claim `sub` corresponda estricta y únicamente al `ADMIN_USERNAME` autorizado, previniendo de este modo cualquier intento de elevación de privilegios mediante la inyección de tokens forjados en otros sistemas.

```
# server.py – Función de verificación con manejo de errores diferenciado (líneas
134-143)
def verify_token(token: str = Depends(oauth2_scheme)):
    try:
        payload = jwt.decode(token, SECRET_KEY, algorithms=[ALGORITHM])
        if payload.get("sub") != ADMIN_USERNAME:
            raise HTTPException(status_code=401, detail="Usuario no válido")
        return payload
    except jwt.ExpiredSignatureError:
        raise HTTPException(status_code=401, detail="Token expirado")
    except jwt.InvalidTokenError:
        raise HTTPException(status_code=401, detail="Token inválido")
```

El manejo diferenciado de excepciones permite al cliente (y a los registros del servidor) distinguir entre un token expirado legítimamente y un token potencialmente falsificado o corrupto.

4.7.1.4. Cobertura de Protección: Endpoints Protegidos por JWT

La siguiente tabla enumera exhaustivamente todos los endpoints del servidor que requieren autenticación JWT mediante la dependencia `Depends(verify_token)`:

Método HTTP	Endpoint	Función del Servidor	Acción Protegida
POST	/api/config	guardar_config()	Guardar configuración visual global (imagen de fondo)
GET	/api/solicitudes	listar_solicitudes()	Listar todas las solicitudes de inclusión recibidas
PUT	/api/solicitudes/{id}	actualizar_estado_solicitud()	Cambiar el estado de una solicitud (pendiente → revisada/aprobada/rechazada)
DELETE	/api/solicitudes/{id}	eliminar_solicitud()	Eliminar permanentemente una solicitud del registro
POST	/api/upload-logo	subir_logo()	Subir una imagen de logotipo de asociación al servidor
POST	/api/upload-fondo	subir_fondo()	Subir un nuevo fondo de pantalla global al sistema
POST	/api/importar	importar_asociaciones()	Importar asociaciones masivamente desde un archivo CSV u ODS
GET	/api/exportar/csv	exportar_csv()	Descargar copia de seguridad completa del catálogo en formato CSV
GET	/api/exportar/ods	exportar_ods()	Descargar copia de seguridad completa del catálogo en formato ODS
POST	/api/{recurso}	escribir_datos()	Crear nuevos registros de asociaciones, categorías o etiquetas
PUT	/api/{recurso}/{id}	actualizar_registro()	Modificar un registro existente de cualquier recurso
DELETE	/api/{recurso}/{id}	eliminar_registro()	Eliminar un registro de cualquier recurso

Los endpoints de **lectura pública** (`GET /api/{recurso}`, `GET /api/config`, `GET /api/fondos`, `POST /api/solicitudes`) permanecen **sin protección** intencionadamente, ya que sirven el catálogo público de asociaciones, listan las imágenes de fondo disponibles y permiten el envío de solicitudes por parte de usuarios anónimos.

4.7.1.5. Gestión de Sesión y Control de Acceso en el Cliente

La capa de cliente implementa mecanismos de control de acceso complementarios. El script `auth.js` verifica la presencia del token en el `localStorage` durante la carga de las páginas de administración. Si el token no existe y la página actual requiere autenticación, el script interrumpe la carga y redirige automáticamente al formulario de login, previniendo el acceso no autorizado a las vistas protegidas.

```
// auth.js – Guardián de rutas administrativas
const token = localStorage.getItem('adminToken');
if (!token && !isLoginPage && !isPublicPage) {
    window.location.href = 'login.html';
    return;
}
```

Adicionalmente, este script controla la visibilidad de los elementos del DOM mediante las clases CSS `.req-admin` y `.req-publico`. Esto asegura que los controles de administración (editar, eliminar o importar registros) permanezcan ocultos en la interfaz pública y solo sean visibles cuando existe una sesión activa válida.

```
// auth.js – Ocultación selectiva de controles administrativos
if (token) {
    document.querySelectorAll('.req-admin').forEach(el => el.style.display = 'block');
    document.querySelectorAll('.req-publico').forEach(el => el.style.display = 'none');
} else {
    document.querySelectorAll('.req-admin').forEach(el => el.style.display = 'none');
    document.querySelectorAll('.req-publico').forEach(el => el.style.display = 'block');
}
```

El cierre de sesión (Logout) se implementa eliminando el token del `localStorage` y redirigiendo a la página principal. Dado que la arquitectura es sin estado (stateless), no se requiere comunicación adicional con el servidor. Además, existe una detección de expiración: si una petición asíncrona recibe una respuesta `HTTP 401` del backend, el cliente asume que el token ha caducado, limpia el `localStorage` y redirige al panel de login para que el usuario pueda reautenticarse.

4.7.2. Inyección de Cabeceras de Seguridad (Security Headers Middleware)

Para proteger al portal contra ataques comunes a nivel de navegador y red, el servidor FastAPI integra un middleware de interceptación que inyecta automáticamente cabeceras HTTP de seguridad robustas en **cada respuesta** emitida por el servidor, independientemente del endpoint solicitado. A continuación se describe en profundidad la arquitectura del middleware y el papel defensivo de cada cabecera.

4.7.2.1. Arquitectura del Middleware Interceptor

El middleware se implementa como una clase de Python que hereda de `BaseHTTPMiddleware` de Starlette. Sigue el patrón *chain-of-responsibility*, interceptando la petición HTTP entrante antes de que alcance el endpoint de la API. El middleware procesa la petición y la delega a la aplicación mediante `call_next(request)`. Una vez que el servidor

genera la respuesta, el middleware la intercepta antes de enviarla al cliente para añadir las cabeceras de seguridad. Finalmente, la respuesta con las cabeceras inyectadas se devuelve al navegador.

```
# server.py – Middleware de Cabeceras de Seguridad (líneas 28-45)
from starlette.middleware.base import BaseHTTPMiddleware
from starlette.responses import Response

class SecurityHeadersMiddleware(BaseHTTPMiddleware):
    async def dispatch(self, request, call_next):
        response: Response = await call_next(request)
        response.headers["X-Frame-Options"] = "DENY"
        response.headers["X-Content-Type-Options"] = "nosniff"
        response.headers["X-XSS-Protection"] = "1; mode=block"
        response.headers["Referrer-Policy"] = "strict-origin-when-cross-origin"
        response.headers["Content-Security-Policy"] = (
            "default-src 'self'; "
            "script-src 'self' 'unsafe-inline' https://fonts.googleapis.com; "
            "style-src 'self' 'unsafe-inline' https://fonts.gstatic.com; "
            "font-src 'self' https://fonts.gstatic.com; "
            "img-src 'self' data: http: https:; "
            "frame-src https://www.youtube.com https://youtube.com;"
        )
        return response

app.add_middleware(SecurityHeadersMiddleware)
```

Este enfoque garantiza que **ninguna respuesta del servidor escape sin las cabeceras de protección**, incluyendo las respuestas a archivos estáticos, endpoints de la API REST, y redirecciones.

4.7.2.2. Tabla Resumen de Cabeceras Inyectadas

Cabecera HTTP	Valor Configurado	Ataque Mitigado	Estándar/Referencia
X-Frame-Options	DENY	Clickjacking	RFC 7034
Content-Security-Policy	Política multi-directiva (ver 4.7.2.4)	XSS, inyección de recursos, exfiltración de datos	W3C CSP Level 2
X-Content-Type-Options	nosniff	MIME Sniffing / Ejecución de código camuflado	Fetch Standard
X-XSS-Protection	1; mode=block	XSS reflejado (navegadores legacy)	No estandarizado (Microsoft)
Referrer-Policy	strict-origin-when-cross-origin	Filtración de URLs internas	W3C Referrer Policy

4.7.2.3. Protección contra Clickjacking (X-Frame-Options: DENY)

Vector de Ataque Mitigado: El *clickjacking* (o *UI redressing*) consiste en que un atacante embebe el portal legítimo dentro de un `<iframe>` transparente superpuesto en su propia página maliciosa. El administrador, creyendo interactuar con la página del atacante, en realidad realiza clics sobre los botones de la aplicación real (eliminar registros, modificar configuración, etc.) sin ser consciente de ello.

Mecanismo de Defensa: La cabecera `X-Frame-Options: DENY` instruye al navegador para que **bloquee por completo** la carga del portal dentro de cualquier `<iframe>`, `<frame>`, `<embed>` u `<object>`, sin importar el origen del sitio que intente embebido.

```
X-Frame-Options: DENY
```

Alcance: Se ha elegido el valor `DENY` (el más restrictivo) en lugar de `SAMEORIGIN` porque la aplicación no requiere ser embebida ni siquiera desde su propio dominio. Esto elimina por completo el vector de clickjacking.

4.7.2.4. Política de Seguridad de Contenido (Content-Security-Policy)

La cabecera `Content-Security-Policy` (CSP) es la capa de defensa más exhaustiva del middleware. Define una **lista blanca de orígenes autorizados** para cada tipo de recurso que el navegador puede cargar, bloqueando cualquier recurso externo no listado explícitamente. Esto mitiga de raíz los ataques de inyección de código, exfiltración de datos y carga de recursos maliciosos.

La política se compone de **6 directivas** configuradas de forma granular para satisfacer las necesidades funcionales del portal sin relajar la seguridad:

Directiva `default-src 'self'`

```
default-src 'self';
```

Esta directiva establece la política por defecto para todos los recursos no cubiertos explícitamente por otras directivas específicas. Su efecto principal es autorizar la carga de recursos (scripts, estilos, fuentes, conexiones AJAX) procedentes del propio origen del servidor (`'self'`). Como resultado, el navegador bloqueará cualquier intento de incrustar o ejecutar recursos desde un dominio externo que no haya sido autorizado previamente, garantizando la seguridad estructural básica.

Directiva `script-src 'self' 'unsafe-inline'`

```
script-src 'self' 'unsafe-inline';
```

Esta directiva controla los orígenes desde los cuales el navegador puede descargar y ejecutar código JavaScript. El parámetro `'self'` autoriza la carga de los archivos `.js` servidos desde el propio servidor (por ejemplo, el directorio `/html/js/`). La inclusión del parámetro `'unsafe-inline'` permite la ejecución de scripts definidos directamente en el HTML (`<script>...</script>`). Este permiso es necesario porque la arquitectura en Vanilla JS emplea inicializaciones en línea; no obstante, el riesgo de inyección se mitiga mediante los mecanismos de sanitización contra ataques XSS implementados en el cliente.

Directiva `style-src 'self' 'unsafe-inline' https://fonts.googleapis.com`

```
style-src 'self' 'unsafe-inline' https://fonts.googleapis.com;
```

Esta directiva controla los orígenes autorizados para la descarga y aplicación de hojas de estilo CSS. El parámetro `'self'` permite aplicar las reglas de estilo locales alojadas en el servidor, como `/html/css/style_Form.css` y `/html/css/style_Index.css`. El parámetro `'unsafe-inline'` habilita la aplicación de estilos en línea (`style="..."`) modificados dinámicamente mediante JavaScript, capacidad necesaria para la previsualización del fondo global, la manipulación de la visibilidad de controles administrativos y la maquetación de tarjetas. Por último, se autoriza explícitamente el dominio `https://fonts.googleapis.com` para permitir al navegador conectarse a la red de distribución de Google Fonts y descargar las familias tipográficas Montserrat y Outfit, como se refleja en la siguiente importación:

```
/* style_Form.css – Importación de fuentes tipográficas */
@import url('https://fonts.googleapis.com/css2?
family=Montserrat:wght@400;500;600;700&family=Outfit:wght@300;400;500;600;700&displ
ay=swap');
```

Directiva `font-src 'self' https://fonts.gstatic.com`

```
font-src 'self' https://fonts.gstatic.com;
```

Complementando la directiva de estilos, esta instrucción controla los orígenes autorizados para descargar archivos binarios de fuentes tipográficas (`.woff2`, `.ttf`). Se habilita el origen `https://fonts.gstatic.com`, que corresponde al CDN de Google encargado de servir los archivos de las fuentes. Mientras que el dominio de Google APIs provee la hoja de estilos CSS, Gstatic transfiere los archivos binarios; ambos son necesarios para renderizar correctamente las tipografías corporativas Montserrat y Outfit.

Directiva `img-src 'self' data: http: https:`

```
img-src 'self' data: http: https;;
```

La directiva `img-src` restringe los orígenes permitidos para cargar imágenes. El origen `'self'` autoriza las imágenes servidas localmente desde el servidor, como logotipos alojados en `/html/img/logos/`. El esquema `data:` permite cargar imágenes codificadas en Base64 insertadas directamente en el HTML, lo cual es necesario para la previsualización de logotipos en el formulario de creación mediante la API `FileReader`. Por último, se permiten los esquemas `http:` y `https:` para admitir logotipos de asociaciones alojados en servidores web externos.

Directiva `frame-src https://www.youtube.com https://youtube.com`

```
frame-src https://www.youtube.com https://youtube.com;
```

Esta directiva restringe los orígenes permitidos para los elementos `<iframe>`. Su función es habilitar la carga segura de la galería multimedia de vídeos en la ficha de cada asociación, donde el script `detalle.js` genera dinámicamente iframes que apuntan a YouTube:

```
// detalle.js – Renderizado de galería de vídeos (líneas 231-237)
const iframe = document.createElement('iframe');
iframe.src = `https://www.youtube.com/embed/${videoId}`;
```

```
iframe.title = 'Vídeo de YouTube';  
iframe.allow = 'accelerometer; autoplay; clipboard-write; encrypted-media;  
gyroscope; picture-in-picture; web-share';  
iframe.allowFullscreen = true;  
iframe.loading = 'lazy';
```

La directiva establece que www.youtube.com y youtube.com son los únicos orígenes válidos para inyectar contenido en iframes. Cualquier intento de cargar un iframe desde un origen no listado será bloqueado por el navegador, previniendo la ejecución de contenido multimedia externo no confiable.

4.7.2.5. Prevención de MIME Sniffing (**X-Content-Type-Options: nosniff**)

Vector de Ataque Mitigado: El *MIME sniffing* es un comportamiento de ciertos navegadores que intentan "adivinar" el tipo real de un recurso ignorando la cabecera **Content-Type** declarada por el servidor. Un atacante podría explotar esto subiendo un archivo con extensión **.jpg** que internamente contenga código JavaScript. Sin esta protección, el navegador podría detectar el contenido como script y ejecutarlo.

Mecanismo de Defensa: La cabecera **X-Content-Type-Options: nosniff** instruye al navegador para que respete **estrictamente** el tipo MIME declarado por el servidor en cada respuesta, sin intentar inferir un tipo alternativo.

```
X-Content-Type-Options: nosniff
```

Relevancia en el proyecto: Esta cabecera es especialmente crítica porque el portal permite la **subida de archivos de imagen** (logotipos de asociaciones vía `/api/upload-logo`) y sirve archivos estáticos desde el directorio `/html/`. Sin **nosniff**, un atacante podría intentar camuflar código ejecutable como una imagen y explotar el MIME sniffing para lograr ejecución de código en el navegador de otro usuario o del administrador.

4.7.2.6. Filtro de XSS Reflejado (**X-XSS-Protection: 1; mode=block**)

Vector de Ataque Mitigado: El *XSS reflejado* (Cross-Site Scripting reflejado) ocurre cuando un atacante inyecta código malicioso en los parámetros de una URL (por ejemplo, en un parámetro de búsqueda o un ID) y la aplicación refleja ese código de vuelta en la respuesta HTML sin sanitizarlo. Si la víctima hace clic en la URL manipulada, el código se ejecuta en su navegador.

Mecanismo de Defensa: La cabecera activa el filtro XSS nativo de ciertos navegadores (principalmente Internet Explorer y versiones legacy de Chrome). El valor **mode=block** indica que, si el filtro detecta un intento de XSS reflejado, debe **bloquear completamente la carga de la página** en lugar de intentar sanitizar el contenido:

```
X-XSS-Protection: 1; mode=block
```

Nota técnica: Aunque los navegadores modernos (Chrome 78+, Firefox, Edge) han **deprecado** esta cabecera en favor de políticas CSP robustas (ya cubiertas en la sección 4.7.2.4), se mantiene como **defensa en profundidad** (*defense-in-depth*) para proteger a usuarios que accedan con navegadores antiguos o sin soporte completo de CSP.

4.7.2.7. Política de Privacidad de Referencias (**Referrer-Policy: strict-origin-when-cross-origin**)

Vector de Ataque Mitigado: Cuando un usuario navega desde el portal hacia un enlace externo (por ejemplo, al hacer clic en la web oficial de una asociación o en su perfil de red social), el navegador incluye por defecto una cabecera **Referer** que revela la URL completa de la página de origen. Esto podría exponer rutas administrativas internas, IDs de sesión en la URL, o la estructura de navegación del panel de administración.

Mecanismo de Defensa: La política **strict-origin-when-cross-origin** establece reglas diferenciadas según el destino:

```
Referrer-Policy: strict-origin-when-cross-origin
```

Escenario de Navegación	Información Compartida
Mismo origen (ej. de /html/detalle.html a /html/index.html)	URL completa (path + query string). Funcionamiento normal de navegación interna.
Origen cruzado con mismo protocolo (ej. de https://portal.hospital.es/admin/... a https://facebook.com/...)	Solo el origen (https://portal.hospital.es), sin incluir el path ni los parámetros de la URL.
Degradación de protocolo (ej. de HTTPS a HTTP)	Nada. Se suprime completamente la cabecera Referer para evitar filtración de datos en canales no cifrados.

Relevancia en el proyecto: El portal contiene numerosos enlaces de contacto hacia sitios externos de las asociaciones (redes sociales, páginas web oficiales). Sin esta política, cada clic en un contacto externo revelaría la URL completa de la ficha de detalle (incluyendo el ID de la asociación) al sitio de destino. Con la política configurada, solo se comparte el dominio del portal, preservando la privacidad de la navegación interna.

4.7.3. Sanitización y Prevención de Scripts entre Sitios (XSS) en el Cliente

En el entorno del navegador, se han adoptado políticas y utilidades preventivas estrictas en los scripts de JavaScript para erradicar vulnerabilidades de inyección de código (*Cross-Site Scripting*). Las medidas cubren los tres vectores clásicos de XSS: **almacenado** (datos maliciosos persistidos en el servidor y renderizados a otros usuarios), **reflejado** (código inyectado en parámetros de URL) y **basado en DOM** (manipulación directa del árbol de elementos HTML). A continuación se describe cada técnica implementada.

4.7.3.1. Manipulación Segura del DOM (**textContent** vs **innerHTML**)

Vector de Ataque Prevenido: Si los datos textuales procedentes de la API (nombres de asociaciones, descripciones, siglas, etiquetas, categorías) se insertan en el DOM usando la propiedad **innerHTML**, un atacante podría almacenar una cadena maliciosa como nombre de asociación (por ejemplo, `<img src=x onerror="alert(document.cookie)">`) que se ejecutaría en el navegador de cada visitante que acceda a la ficha.

Técnica Defensiva: En todas las vistas públicas y administrativas se ha establecido como **norma de desarrollo** el uso exclusivo de la propiedad **textContent** (o **innerText**) para insertar datos textuales procedentes del servidor. Esta propiedad trata **todo el contenido como texto plano**, escapando automáticamente cualquier carácter HTML especial (`<`, `>`, `&`, `"`, `'`) e impidiendo que se interprete como marcado ejecutable.

Ejemplo concreto — Renderizado de la ficha de detalle de una asociación (**detalle.js**):

```
// detalle.js – Inserción segura de datos textuales (líneas 114-122)
// Textos directos seguros – NUNCA se usa innerHTML para estos datos
```

```

detalleNombre.textContent = asoc.nombre_asociacion;
detalleSiglas.textContent = asoc.siglas;
detalleDesc.textContent = asoc.descripcion;
detalleServicios.textContent = asoc.cartera_servicios;

// Resolver Categoría
const catObj = categorias.find(c => c.id === asoc.categoria);
detalleCategoria.textContent = catObj ? catObj.nombre : 'No especificada';

```

Ejemplo concreto — Renderizado de tarjetas en el catálogo público (`index.js`):

```

// index.js – Construcción segura de tarjetas (líneas 196-212)
const h3 = document.createElement('h3');
h3.textContent = asoc.nombre_asociacion;           // Texto plano, no HTML

const divSiglas = document.createElement('div');
divSiglas.textContent = asoc.siglas;               // Texto plano, no HTML

// Etiquetas como insignias individuales
const tagSpan = document.createElement('span');
tagSpan.textContent = etiObj.nombre;               // Texto plano, no HTML

```

Patrón general aplicado: Todos los datos dinámicos procedentes de la API que representan texto informativo (nombres, siglas, descripciones, categorías, etiquetas, mensajes de error, estados de solicitudes) se insertan con `textContent`. El uso de `innerHTML` se reserva **exclusivamente** para el marcado estático controlado por el desarrollador (iconos SVG predefinidos en código, estructuras HTML fijas), nunca para datos introducidos por el usuario.

4.7.3.2. Sanitización de URLs — Función `safeUrl()`

Vector de Ataque Prevenido: Un atacante podría registrar una asociación con una URL de logo o contacto que use el pseudoprotocolo `javascript:` (por ejemplo, `javascript:alert(document.cookie)`). Cuando el navegador renderice un `` o un `<a href="javascript:...">`, el código malicioso se ejecutaría en el contexto de la sesión del usuario víctima.

Técnica Defensiva: Se ha implementado una función validadora `safeUrl(url)` replicada en los dos scripts de renderizado público (`index.js` y `detalle.js`) que analiza cada URL antes de asignarla a un atributo `src` o `href`:

```

// index.js (líneas 49-57) y detalle.js (líneas 64-72) – Sanitización de URLs
function safeUrl(url) {
  if (!url) return '';
  const cleanUrl = url.trim();
  if (cleanUrl.toLowerCase().startsWith('javascript:')) {
    console.warn('URL bloqueada por seguridad (intento de inyección javascript:)', cleanUrl);
    return '';
  }
  return cleanUrl;
}

```

Proceso de validación:

- 1. **Control de nulidad:** Si la URL es vacía, `null` o `undefined`, devuelve una cadena vacía segura.
- 2. **Eliminación de espacios:** `trim()` elimina espacios en blanco al inicio y al final, previniendo evasiones como `javascript:alert(1)`.
- 3. **Detección de protocolo malicioso:** La comprobación `toLowerCase().startsWith('javascript:')` detecta el pseudoprotocolo `javascript:` con independencia de mayúsculas/minúsculas (`JavaScript:`, `JAVASCRIPT:`, `jAvAsCrIpT:`, etc.).
- 4. **Bloqueo y registro:** Si se detecta un intento, la URL se neutraliza devolviendo una cadena vacía y se emite un aviso en la consola para depuración/auditoría.

Puntos de aplicación:

Archivo	Línea	Uso
index.js	190	<code>img.src = safeUrl(asoc.logo)</code> — Carga de logotipos en las tarjetas del catálogo
detalle.js	108	<code>detalleLogo.src = safeUrl(asoc.logo)</code> — Carga del logo en la ficha de detalle
detalle.js	193	<code>a.href = safeUrl(cleanVal)</code> — Renderizado de URLs de contacto (webs, redes sociales)

4.7.3.3. Escapado de HTML en Atributos — Funciones `escapeHTML()` y `escapeHTMLSol()`

Vector de Ataque Prevenido: En los formularios de administración, cuando se precargan los datos existentes de una asociación para editarla, los valores del servidor se insertan en atributos `value` de inputs HTML mediante *template literals*. Si un dato almacenado contuviera caracteres especiales HTML (por ejemplo, un nombre de contacto como `<script>alert(1)</script>`), el atacante podría romper el atributo e inyectar código ejecutable.

Técnica Defensiva: Se han implementado dos funciones de escapado de caracteres HTML que reemplazan los 5 caracteres peligrosos por sus entidades HTML equivalentes:

```
// crearEntrada.js – escapeHTML para formularios de edición (líneas 139-150)
function escapeHTML(str) {
  if (!str) return '';
  return str.replace(/&<>'"/g,
    tag => ({
      '&': '&amp;',
      '<': '&lt;',
      '>': '&gt;',
      '"': '&#39;',
      "'": '&quot;'
    }[tag] || tag)
  );
}
```

```
// crearEntrada.js – escapeHTMLSol para solicitudes de inclusión (línea 1031-1034)
function escapeHTMLSol(str) {
  if (!str) return '';
  return String(str).replace(/&<>'"/g, tag =>
    ({ '&': '&amp;', '<': '&lt;', '>': '&gt;', '"': '&quot;', "'": '&#39;' }
  [tag] || tag)
  );
}
```

Tabla de caracteres escapados:

Carácter Original	Entidad HTML	Ataque Prevenido
&	&	Inicio de entidades HTML inesperadas
<	<	Apertura de etiquetas HTML/script
>	>	Cierre de etiquetas HTML
"	"	Ruptura de atributos entre comillas dobles
'	'	Ruptura de atributos entre comillas simples

En la práctica, la función `escapeHTML()` se utiliza durante la precarga de datos en los formularios de administración. Se aplica al insertar variables en los atributos HTML, protegiendo campos de entrada como información de contacto y enlaces a vídeos:

```
// crearEntrada.js – Precarga segura de contactos en modo edición (líneas 107-108)
`<input type="text" name="otro_tipo_contacto[]" value="${escapeHTML(customValue)}"
...>
  <input type="text" name="valor_contacto[]" value="${escapeHTML(valor)}" ...>`
```

La función `escapeHTMLSol()` protege la interfaz administrativa al renderizar las tarjetas de solicitudes de inclusión recibidas desde el exterior. Dado que estos datos proceden de usuarios anónimos (nombre de asociación, siglas, descripción, etc.), su sanitización antes de insertarse en el DOM es imprescindible:

```
// crearEntrada.js – Renderizado seguro de solicitudes (líneas 935-962)
`<h3 class="solicitud-nombre">${escapeHTMLSol(sol.nombre_asociacion)}</h3>
  <span class="solicitud-siglas">${escapeHTMLSol(sol.siglas)}</span>
  <p class="solicitud-desc">${escapeHTMLSol(sol.descripcion)}</p>
  <strong>${escapeHTMLSol(sol.representante?.nombre || '')}</strong>`
```

4.7.3.4. Codificación URI Segura en Direcciones de Correo

Vector de Ataque Prevenido: Un atacante podría almacenar una dirección de correo electrónico maliciosa que al renderizarse en un enlace `mailto:` inyecte cabeceras SMTP adicionales o código JavaScript. Por ejemplo, `test@evil.com%0D%0ABcc:spam@attacker.com` podría añadir destinatarios ocultos al correo.

Técnica Defensiva: En la ficha de detalle (`detalle.js`), las direcciones de correo electrónico se codifican con `encodeURIComponent()` antes de insertarse en el atributo `href` del enlace `mailto:`. Esta función convierte todos los caracteres especiales (saltos de línea, signos de porcentaje, espacios, etc.) en secuencias de escape seguras:

```
// detalle.js – Codificación segura de mailto (líneas 186-190)
if (cleanTipo === 'email') {
  const a = document.createElement('a');
  a.href = `mailto:${encodeURIComponent(cleanVal)}`; // Codificación URI segura
  a.textContent = cleanVal; // Texto plano visible
  li.appendChild(a);
}
```

Simultáneamente, el texto visible del enlace se renderiza con `textContent`, garantizando que el usuario vea la dirección original sin interpretar caracteres especiales como HTML.

4.7.3.5. Aislamiento de Enlaces Externos (`rel="noopener noreferrer"`)

Vector de Ataque Prevenido: Cuando un usuario hace clic en un enlace externo que se abre en una nueva pestaña (`target="_blank"`), la página de destino puede acceder al objeto `window.opener` de la página de origen. Un sitio malicioso podría usar `window.opener.location = 'https://phishing.com'` para redirigir silenciosamente la pestaña original del portal a un sitio de *phishing* que imite el formulario de login del administrador.

Técnica Defensiva: Todos los enlaces externos generados dinámicamente en el portal incluyen el atributo `rel="noopener noreferrer"`, que anula por completo el acceso al objeto `window.opener` desde la página de destino:

```
// detalle.js – Aislamiento de enlaces externos (líneas 191-197)
} else if (['web', 'linkedin', 'facebook', 'instagram', 'x',
'tiktok'].includes(cleanTipo)
    || cleanVal.startsWith('http://') || cleanVal.startsWith('https://')) {
    const a = document.createElement('a');
    a.href = safeUrl(cleanVal);           // URL sanitizada
    a.target = '_blank';                  // Nueva pestaña
    a.rel = 'noopener noreferrer';        // Aislamiento de contexto
    a.textContent = cleanVal;             // Texto plano, no HTML
    li.appendChild(a);
}
```

El uso de este atributo cumple dos funciones de seguridad. La propiedad `noopener` impide que la nueva pestaña tenga acceso al objeto `window.opener` de la pestaña original, mitigando vulnerabilidades como el tabnapping. La propiedad `noreferrer` elimina la cabecera `Referer` en la petición HTTP externa; esto complementa la cabecera `Referrer-Policy` y evita que el sitio de destino rastree la URL interna del portal desde la cual se realizó la solicitud.

4.7.3.6. Tabla Resumen: Cobertura de Protección XSS por Archivo

Archivo	Técnica Aplicada	Datos Protegidos
index.js	textContent	Nombres, siglas, etiquetas de tarjetas del catálogo público
index.js	safeUrl()	URLs de logotipos en tarjetas
detalle.js	textContent	Nombre, siglas, descripción, cartera de servicios, categoría, etiquetas
detalle.js	safeUrl()	Logo de asociación, enlaces de contacto (web, redes sociales)
detalle.js	encodeURIComponent()	Direcciones de correo electrónico en enlaces <code>mailto:</code>
detalle.js	rel="noopener noreferrer"	Todos los enlaces externos de contacto
crearEntrada.js	escapeHTML()	Valores precargados en inputs de edición (contactos, vídeos)
crearEntrada.js	escapeHTMLSol()	Datos de solicitudes de inclusión (nombre, siglas, descripción, representante, web, mensaje)

Archivo	Técnica Aplicada	Datos Protegidos
contacto.js	textContent	Mensajes de error, contadores, datos de confirmación
login.js	textContent	Mensajes de error de autenticación

4.7.4. Validación de Esquemas en la Frontera (Inmunidad del Almacenamiento)

Cualquier dato enviado hacia el servidor pasa obligatoriamente por una validación estricta de tipos y campos en la frontera de FastAPI **antes de tocar el almacenamiento físico** JSON en disco. Este principio de *"inmunidad del almacenamiento"* garantiza que los archivos de datos (`asociaciones.json`, `categorias.json`, `etiquetas.json`, `solicitudes.json`, `config.json`) solo contengan datos estructuralmente válidos, impidiendo la corrupción de la base de datos y bloqueando de raíz payloads maliciosos. A continuación se describen todos los mecanismos de validación implementados.

4.7.4.1. Catálogo de Modelos Pydantic

El servidor define **8 modelos de datos** con la librería Pydantic que actúan como contratos estrictos de validación. Cada modelo especifica los campos requeridos, sus tipos de datos y sus valores por defecto:

Modelo `Contacto` — Método de contacto individual

```
class Contacto(BaseModel):
    tipo: str          # email, linkedin, web, telefono, x, instagram, facebook,
    tiktok...
    valor: str         # URL, dirección de correo, número de teléfono...
```

Campo	Tipo	Obligatorio	Descripción
tipo	str	☑ Sí	Identificador del tipo de medio de contacto
valor	str	☑ Sí	Valor concreto del contacto (URL, email, teléfono)

Modelo `Ubicacion` — Localización geográfica

```
class Ubicacion(BaseModel):
    pais: Optional[str] = ""
    comunidad: Optional[str] = ""
    provincia: Optional[str] = ""
    municipio: Optional[str] = ""
```

Campo	Tipo	Obligatorio	Valor por Defecto
pais	Optional[str]	✗ No	"" (cadena vacía)
comunidad	Optional[str]	✗ No	""
provincia	Optional[str]	✗ No	""
municipio	Optional[str]	✗ No	""

Modelo `Asociacion` — Registro principal del catálogo

```
class Asociacion(BaseModel):
    id: Union[int, str]
    nombre_asociacion: str
    siglas: str
    logo: Optional[str] = ""
    descripcion: str
    categoria: str
    etiquetas: List[str]
    contactos: List[Contacto]
    cartera_servicios: str
    videos: Optional[List[str]] = []
    ubicacion: Optional[Ubicacion] = None
```

Campo	Tipo	Obligatorio	Validación Implícita
id	Union[int, str]	✔ Sí	Acepta enteros (timestamps) o cadenas
nombre_asociacion	str	✔ Sí	No puede ser nulo ni omitido
siglas	str	✔ Sí	No puede ser nulo ni omitido
logo	Optional[str]	✗ No	URL del logo o cadena vacía
descripcion	str	✔ Sí	No puede ser nulo ni omitido
categoria	str	✔ Sí	ID de categoría (referencia cruzada)
etiquetas	List[str]	✔ Sí	Lista de IDs de etiquetas; Pydantic rechaza tipos no-lista
contactos	List[Contacto]	✔ Sí	Lista de subobjetos; cada elemento se valida recursivamente contra el modelo Contacto
cartera_servicios	str	✔ Sí	No puede ser nulo ni omitido
videos	Optional[List[str]]	✗ No	Lista de URLs/IDs de YouTube
ubicacion	Optional[Ubicacion]	✗ No	Subobjeto validado recursivamente o None

Modelos **Categoria** y **Etiqueta** — Taxonomía del directorio

```
class Categoria(BaseModel):
    id: str
    nombre: str

class Etiqueta(BaseModel):
    id: str
    nombre: str
```

Campo	Tipo	Obligatorio
id	str	✔ Sí
nombre	str	✔ Sí

Modelo **ConfigWeb** — Configuración visual global

```
class ConfigWeb(BaseModel):
    fondo: Optional[str] = ""
```

Campo	Tipo	Obligatorio	Valor por Defecto
fondo	Optional[str]	✗ No	"" (sin imagen de fondo)

Modelos **RepresentanteSolicitud** y **SolicitudAsociacion** — Solicitudes de inclusión

```
class RepresentanteSolicitud(BaseModel):
    nombre: str
    cargo: str
    email: str
    telefono: Optional[str] = ""

class SolicitudAsociacion(BaseModel):
    nombre_asociacion: str
    siglas: str
    descripcion: str
    categoria_sugerida: Optional[str] = ""
    web: Optional[str] = ""
    ubicacion: Optional[Ubicacion] = None
    mensaje: Optional[str] = ""
    representante: RepresentanteSolicitud
```

Campo (SolicitudAsociacion)	Tipo	Obligatorio	Validación Implícita
nombre_asociacion	str	✓ Sí	No puede ser nulo ni omitido
siglas	str	✓ Sí	No puede ser nulo ni omitido
descripcion	str	✓ Sí	No puede ser nulo ni omitido
categoria_sugerida	Optional[str]	✗ No	Cadena libre sugerida por el solicitante
web	Optional[str]	✗ No	URL de la asociación
ubicacion	Optional[Ubicacion]	✗ No	Subobjeto validado recursivamente
mensaje	Optional[str]	✗ No	Mensaje adicional del solicitante
representante	RepresentanteSolicitud	✓ Sí	Subobjeto obligatorio con campos nombre , cargo y email requeridos

4.7.4.2. Validación Automática en Firma de Endpoint (FastAPI + Pydantic)

Para los endpoints que reciben payloads con estructura predecible, FastAPI aplica la validación Pydantic **automáticamente en la firma de la función**, antes de que se ejecute una sola línea de lógica de negocio:

```
# server.py – Validación automática en la firma (líneas 170-176 y 196-206)
@app.post("/api/config")
async def guardar_config(payload: ConfigWeb, token: dict = Depends(verify_token)):
```



```
# Si payload no cumple ConfigWeb → FastAPI devuelve 422 automáticamente
# Esta línea SOLO se ejecuta si la validación fue exitosa
...

@app.post("/api/solicitudes")
async def crear_solicitud(payload: SolicitudAsociacion):
    # Si payload no cumple SolicitudAsociacion → FastAPI devuelve 422
    automáticamente
    # Validación recursiva: representante.nombre, representante.email, etc.
    ...
```

Comportamiento de rechazo automático: Si el cuerpo de la petición no cumple con el esquema del modelo, FastAPI intercepta la petición y devuelve un error **422 Unprocessable Content** con un informe detallado de cada campo que falló la validación:

```
{
  "detail": [
    {
      "loc": ["body", "representante", "email"],
      "msg": "field required",
      "type": "value_error.missing"
    }
  ]
}
```

Este mecanismo protege especialmente el endpoint público **POST /api/solicitudes**, donde cualquier usuario anónimo puede enviar datos. Sin la validación Pydantic, un atacante podría inyectar estructuras JSON arbitrarias que corromperían **solicitudes.json**.

4.7.4.3. Validación Explícita con Bloque **try/except** (Endpoints Genéricos)

Para los endpoints genéricos que manejan múltiples tipos de recurso (**POST /api/{recurso}**, **PUT /api/{recurso}/{id}**), la validación Pydantic se aplica **explícitamente** dentro de un bloque **try/except** **ValidationError**, ya que el modelo concreto se selecciona dinámicamente en función del recurso solicitado:

```
# server.py – Validación explícita en POST genérico (líneas 459-477)
if isinstance(payload, dict):
    try:
        if recurso == 'asociaciones':
            validated_item = Asociacion(**payload)           # Validación completa
            existing_data.append(validated_item.dict())       # Solo si pasa
        elif recurso == 'categorias':
            validated_item = Categoria(**payload)
            existing_data.append(validated_item.dict())
        elif recurso == 'etiquetas':
            validated_item = Etiqueta(**payload)
            existing_data.append(validated_item.dict())
    except ValidationError as e:
        raise HTTPException(status_code=400, detail=f"Datos malformados: {e.errors()}")
```

```
# server.py – Validación explícita en PUT genérico (líneas 519-530)
try:
    if recurso == 'asociaciones':
        validated_item = Asociacion(**payload)
        existing_data[found_index] = validated_item.dict()
    elif recurso == 'categorias':
        validated_item = Categoria(**payload)
        existing_data[found_index] = validated_item.dict()
    elif recurso == 'etiquetas':
        validated_item = Etiqueta(**payload)
        existing_data[found_index] = validated_item.dict()
except ValidationError as e:
    raise HTTPException(status_code=400, detail=f"Datos de actualización
malformados: {e.errors()}")
```

Patrón de seguridad crítico: La función `write_data()` que persiste los datos en disco **solo se invoca después** de que la validación haya completado sin errores. Si el constructor del modelo Pydantic (`Asociacion(**payload)`) lanza una `ValidationError`, la excepción se captura, se transforma en un error HTTP **400 Bad Request**, y la operación de escritura **nunca se ejecuta**. Esto garantiza que un payload corrupto jamás alcance el almacenamiento.

Caso de sobrescritura en bloque: Cuando el endpoint recibe una lista completa de registros, **cada elemento individual** se valida contra su modelo correspondiente antes de permitir la escritura. Si un solo elemento de la lista es inválido, toda la operación se aborta:

```
# server.py – Validación elemento a elemento en modo bloque (líneas 483-494)
elif isinstance(payload, list):
    validated_list = []
    try:
        for item in payload:
            if recurso == 'asociaciones':
                validated_list.append(Asociacion(**item).dict())
            elif recurso == 'categorias':
                validated_list.append(Categoria(**item).dict())
            elif recurso == 'etiquetas':
                validated_list.append(Etiqueta(**item).dict())
    except ValidationError as e:
        raise HTTPException(status_code=400, detail=f"Lista de datos malformados:
{e.errors()}")
```

4.7.4.4. Protección de Identidad (Inmutabilidad de IDs)

En el endpoint `PUT /api/{recurso}/{id}`, después de localizar el registro existente, el servidor **sobrescribe el campo `id` del payload con el ID original** almacenado en la base de datos, impidiendo que un cliente malintencionado mute el identificador de un registro durante una actualización:

```
# server.py – Conservación forzada del ID original (líneas 516-517)
# Conservar el ID original para evitar mutaciones de ID en BD
payload["id"] = existing_data[found_index]["id"]
```

Sin esta medida, un atacante autenticado podría enviar un payload con un `id` diferente al de la URL, generando duplicados o sobreescrituras cruzadas que corromperían la integridad referencial del directorio.

4.7.4.5. Restricción de Recursos (Whitelist de Nombres de Recurso)

Antes de ejecutar cualquier operación de lectura o escritura, la función auxiliar `get_resource_file()` valida que el nombre del recurso solicitado pertenezca a una **lista blanca cerrada** de valores aceptados:

```
# server.py – Whitelist de recursos (líneas 104-107)
def get_resource_file(recurso: str) -> str:
    if recurso not in ['asociaciones', 'categorias', 'etiquetas']:
        raise HTTPException(status_code=404, detail="Recurso no encontrado")
    return f"data/{recurso}.json"
```

Ataque prevenido: Sin esta validación, un atacante podría intentar acceder o manipular archivos arbitrarios del sistema mediante la inyección de rutas en el parámetro `{recurso}` (por ejemplo, `../config` o `../../etc/passwd`). La whitelist limita las operaciones exclusivamente a los tres archivos de datos legítimos del directorio.

4.7.4.6. Validación de Archivos Subidos (Extensión y Tamaño)

El endpoint `POST /api/upload-logo` implementa una doble barrera de validación para los archivos de imagen subidos al servidor:

Barrera 1 — Validación de extensión contra whitelist:

```
# server.py – Extensiones de imagen permitidas (líneas 237-251)
ALLOWED_LOGO_EXTENSIONS = {'.jpg', '.jpeg', '.png', '.gif', '.webp', '.svg'}

ext = os.path.splitext(original_name)[1].lower()
if ext not in ALLOWED_LOGO_EXTENSIONS:
    raise HTTPException(
        status_code=400,
        detail=f"Tipo de archivo no permitido. Solo se aceptan: {'',
'.join(ALLOWED_LOGO_EXTENSIONS)}"
    )
```

Extensión Permitida	Formato
.jpg, .jpeg	JPEG (fotografías y logos rasterizados)
.png	PNG (logos con transparencia)
.gif	GIF (logos animados)
.webp	WebP (formato moderno optimizado)
.svg	SVG (logos vectoriales escalables)

Cualquier extensión no listada (`.exe`, `.php`, `.js`, `.html`, `.bat`, etc.) es rechazada con un error `400 Bad Request`, bloqueando intentos de subir scripts ejecutables camuflados como imágenes.

Barrera 2 — Límite de tamaño máximo de archivo:

```
# server.py — Restricción de tamaño (líneas 260-264)
MAX_SIZE = 5 * 1024 * 1024 # 5 MB
content = await file.read()
if len(content) > MAX_SIZE:
    raise HTTPException(status_code=400, detail="El archivo supera el tamaño máximo permitido (5 MB).")
```

Este límite de **5 MB** previene ataques de **denegación de servicio (DoS)** por agotamiento de disco, donde un atacante podría inundar el servidor con archivos masivos hasta saturar el espacio de almacenamiento.

Barrera 3 — Renombrado con UUID aleatorio:

```
# server.py — Nombre único anti-colisión (línea 254)
unique_name = f"{uuid.uuid4().hex}{ext}"
```

El nombre original del archivo subido es descartado y reemplazado por un identificador UUID v4 hexadecimal aleatorio. Esta medida previene colisiones de nombres, asegurando que los archivos no se sobrescriban accidentalmente en el servidor. Al renombrar el archivo, también se mitigan ataques de escalada de directorios (Path Traversal), ya que cualquier ruta maliciosa inyectada en el nombre del archivo original queda neutralizada. Adicionalmente, el UUID previene la enumeración o predicción de rutas públicas de las imágenes almacenadas.

4.7.4.7. Validación Semántica de Estados (Solicitudes de Inclusión)

El endpoint `PUT /api/solicitudes/{solicitud_id}` implementa una validación semántica adicional que restringe los valores aceptados para el campo `estado` a un conjunto cerrado de valores legítimos:

```
# server.py — Validación de estados válidos (líneas 219-221)
estado = payload.get("estado", sol["estado"])
if estado not in ["pendiente", "revisada", "aprobada", "rechazada"]:
    raise HTTPException(status_code=400, detail="Estado no válido")
```

Estado Válido	Significado
pendiente	Solicitud recibida, pendiente de revisión
revisada	El administrador ha examinado la solicitud
aprobada	Solicitud aceptada para inclusión en el directorio
rechazada	Solicitud denegada

Cualquier valor fuera de este conjunto (por ejemplo, `eliminada`, `<script>alert(1)</script>`, o un campo vacío) es rechazado con un error `400 Bad Request`, impidiendo la corrupción de la máquina de estados del flujo de solicitudes.

4.7.4.8. Tabla Resumen: Cobertura de Validación por Endpoint

Endpoint	Método	Tipo de Validación	Modelo/Mecanismo	Código de Error
/api/config	POST	Automática en firma	ConfigWeb	422
/api/solicitudes	POST	Automática en firma	SolicitudAsociacion → RepresentanteSolicitud → Ubicacion	422
/api/solicitudes/{id}	PUT	Semántica (whitelist de estados)	Comparación contra lista cerrada	400
/api/upload-logo	POST	Extensión + tamaño + renombrado	ALLOWED_LOGO_EXTENSIONS + MAX_SIZE + uuid4	400
/api/importar	POST	Explícita por fila CSV	Asociacion por cada fila procesada	400
/api/{recurso}	POST	Explícita con try/except	Asociacion / Categoria / Etiqueta según recurso	400
/api/{recurso}	POST (lista)	Explícita elemento a elemento	Cada ítem validado individualmente	400
/api/{recurso}/{id}	PUT	Explícita + inmutabilidad de ID	Modelo según recurso + sobrescritura de ID	400
/api/{recurso}	GET/POST/PUT/DELETE	Whitelist de recurso	get_resource_file() → solo asociaciones, categorias, etiquetas	404

4.7.5. Protección Antispam (Filtro Honeypot) en Solicitudes Públicas

Para salvaguardar los recursos del servidor y la base de datos de solicitudes (`solicitudes.json`) contra ataques automatizados de bots (inundación de solicitudes de inclusión falsas o *spam*), se ha implementado un mecanismo de defensa pasivo conocido como **Filtro Honeypot** (tarro de miel) en la capa de cliente. Este enfoque proporciona una alta eficacia de filtrado sin imponer fricción alguna (como los engorrosos CAPTCHAs) al usuario humano.

4.7.5.1. Implementación de la Trampa Silenciosa (HTML)

El formulario público de contacto (`contacto.html`) incorpora un campo de entrada camuflado estratégicamente para atraer a los scripts automatizados:

```
<!-- contacto.html – Campo honeypot anti-spam (oculto) (Líneas 220-221) -->
<input type="text" name="website_url" id="honeypot" tabindex="-1"
autocomplete="off"
style="display:none; visibility:hidden; position:absolute; left:-9999px;">
```

El campo oculto honeypot se basa en cuatro mecanismos. Primero, utiliza el atributo `name="website_url"` como señuelo para atraer a los scripts automatizados que buscan inyectar enlaces. Segundo, el campo se oculta visualmente mediante reglas CSS (`display:none`, `visibility:hidden`, `position:absolute`, `left:-9999px`), asegurando que no sea visible para los usuarios humanos. Tercero, se aplica el atributo `tabindex="-1"` para evitar que los usuarios accedan accidentalmente al campo al navegar mediante el teclado. Finalmente, se incluye el atributo `autocomplete="off"` para impedir que los gestores de contraseñas o el autocompletado del navegador introduzcan datos inadvertidamente en el campo.

4.7.5.2. Comportamiento de los Bots Automatizados

Para los usuarios humanos, este campo es invisible e inaccesible y permanece vacío. En cambio, los bots automatizados y rastreadores procesan el código HTML directamente. Al encontrar la etiqueta de entrada de texto `<input type="text">` con el nombre `website_url`, tienden a rellenarla automáticamente con datos comerciales, activando así el mecanismo de detección.

4.7.5.3. Intercepción y Bloqueo en Cliente (JavaScript)

La validación del honeypot se realiza asíncronamente en el lado del cliente (en el archivo `contacto.js`), exactamente en el momento en que se intenta enviar el formulario, **antes** de realizar ninguna petición de red hacia el backend:

```
// contacto.js – Intercepción del formulario (Líneas 195-204)
form.addEventListener('submit', async (e) => {
  e.preventDefault();

  // Comprobación honeypot anti-spam
  const honeypot = document.getElementById('honeypot');
  if (honeypot && honeypot.value.trim() !== '') {
    console.warn('Posible bot detectado. Envío bloqueado.');
```

```
    return; // Aborta la ejecución de forma silenciosa
  }

  // Si está vacío, continúa con la validación de campos reales...
  if (!validarPaso2()) return;
  // ... envío del payload al servidor
});
```

4.7.5.4. Beneficios del Diseño Honeypot

La implementación del Honeypot aporta varias ventajas. En primer lugar, mejora la experiencia del usuario, ya que no introduce elementos de fricción como los CAPTCHAs y permite al solicitante enviar los datos de manera transparente. En segundo lugar, al interceptar las peticiones en el cliente con JavaScript, se evitan envíos de red innecesarios, ahorrando ancho de banda. Por último, protege los recursos de procesamiento en el servidor, evitando que FastAPI y el sistema de validación Pydantic consuman ciclos de CPU en el procesamiento y rechazo de solicitudes de spam automatizado.

5. IMPLEMENTACIÓN

5.1. Introducción

En este capítulo se describe el proceso de implementación del portal web para el Hospital Universitario San Cecilio. El objetivo de esta fase ha sido trasladar las especificaciones funcionales y técnicas del diseño arquitectónico a código

fuente operativo. El desarrollo se ha llevado a cabo de manera modular, dividiendo la aplicación en tres capas principales: la capa de presentación (Frontend), la capa de lógica de negocio (Backend) y la capa de almacenamiento de datos. Esta separación garantiza un código más mantenible, escalable y con un bajo nivel de acoplamiento.

5.2. Capa de Presentación (Frontend)

El entorno de usuario se ha desarrollado con el objetivo de proporcionar una interfaz clara e intuitiva tanto para el usuario público como para el administrador. Se ha optado por un enfoque basado en tecnologías web estándar, evitando el uso de frameworks de JavaScript complejos como React o Angular, con el fin de optimizar el rendimiento, reducir el peso de las descargas iniciales y simplificar el mantenimiento futuro.

Para la estructura de los datos se ha empleado HTML5, garantizando una correcta semántica, accesibilidad para lectores de pantalla y un posicionamiento orgánico adecuado en motores de búsqueda. Se han utilizado etiquetas estructurales estándar (`<header>`, `<main>`, `<section>`, `<article>`) para definir las distintas áreas de la aplicación. Esta organización semántica es fundamental en el renderizado del catálogo, donde cada asociación se presenta mediante una estructura modular y coherente. Asimismo, se han empleado validaciones nativas de HTML5 en los formularios para establecer una primera capa de control sobre la entrada de datos por parte del usuario.

El diseño visual se gestiona de forma centralizada mediante hojas de estilo CSS3. Se ha implementado un archivo principal que define variables corporativas, estandarizando los colores (verde institucional), tipografías y sombras. Para la disposición de los elementos en pantalla, se ha hecho un uso extensivo de CSS Grid y Flexbox. Estas tecnologías permiten que la interfaz se adapte de forma responsiva a cualquier resolución de pantalla, desde monitores de escritorio hasta dispositivos móviles, reorganizando los componentes automáticamente. Adicionalmente, se han añadido transiciones y efectos de interacción (como sombras y cambios de color al pasar el cursor) que mejoran la experiencia de usuario de manera sutil y sin afectar el rendimiento de renderizado.

La interactividad de la plataforma recae en scripts desarrollados en Vanilla JavaScript. Esta capa es responsable de capturar los eventos del usuario, gestionar el filtrado dinámico del catálogo, abrir y cerrar ventanas modales, y validar formularios antes de su envío. Para la comunicación con el servidor, JavaScript utiliza la Fetch API de forma asíncrona, lo que permite enviar y recibir datos en formato JSON sin necesidad de recargar la página. Además, desde el cliente se gestiona el token de sesión (JWT), almacenándolo en el `localStorage` e interceptando las respuestas del servidor para detectar expiraciones de sesión de forma proactiva.

5.3. Capa de Negocio (Backend)

La lógica de negocio reside en un servidor backend desarrollado en Python, diseñado para ofrecer una API RESTful asíncrona de alto rendimiento. Esta capa actúa como intermediario entre el cliente y los archivos de datos, validando las peticiones y garantizando la seguridad del sistema.

Para el desarrollo del servidor se ha utilizado el framework FastAPI, elegido por su alto rendimiento y soporte nativo para operaciones asíncronas. FastAPI se ejecuta sobre el servidor ASGI Uvicorn, lo que permite manejar múltiples peticiones concurrentes de manera eficiente. Este entorno procesa todas las solicitudes HTTP, gestiona el enrutamiento y aplica funciones middleware para inyectar cabeceras de seguridad en cada respuesta.

Un componente crítico en esta capa es la validación de datos mediante la librería Pydantic. Se han definido modelos de datos estrictos para cada entidad del sistema (asociaciones, categorías, solicitudes). Antes de que cualquier petición de escritura o modificación acceda a la lógica de persistencia, Pydantic verifica que la estructura y los tipos de datos recibidos coincidan exactamente con el esquema esperado. Si se detecta alguna anomalía, el servidor rechaza automáticamente la petición con un código HTTP 422 o 400, protegiendo la base de datos frente a información malformada o maliciosa.

El control de acceso a los endpoints de administración se gestiona mediante tokens JWT (JSON Web Tokens). Al iniciar sesión, el servidor valida las credenciales y emite un token firmado criptográficamente con una vigencia limitada. Este token debe incluirse en la cabecera `Authorization` de todas las peticiones posteriores que requieran privilegios de

administrador. El backend descifra y verifica la firma del token en cada solicitud; si el token es inválido o ha expirado, la operación se aborta devolviendo un error HTTP 401.

5.4. Capa de Datos (Almacenamiento Estructurado)

La capa de datos se ha implementado mediante un sistema de almacenamiento basado en archivos de texto plano en formato JSON, descartando el uso de bases de datos relacionales tradicionales para mantener la simplicidad y portabilidad del proyecto.

El acceso y manipulación de estos archivos se realiza mediante los módulos estándar de entrada/salida de Python. Para asegurar la integridad de la información durante las operaciones de escritura, el servidor carga el archivo en memoria, aplica las modificaciones correspondientes sobre los objetos JSON (asignando identificadores únicos UUID cuando es necesario) y vuelve a escribir el documento completo en disco. Este proceso incluye el manejo de excepciones para evitar la corrupción de archivos en caso de interrupciones inesperadas.

Además de los archivos de texto, la capa de datos gestiona el almacenamiento físico de imágenes. Cuando se recibe un logotipo a través del formulario de administración, el servidor valida el peso y la extensión del archivo. Para prevenir colisiones y problemas de seguridad (como ataques de path traversal), el archivo se renombra utilizando un UUID aleatorio antes de ser guardado definitivamente en el directorio de imágenes.

5.5. Despliegue y Puesta en Marcha

Guía de Despliegue en Servidor — Catálogo de Asociaciones

AsociAcción — Directorio de Asociaciones del Hospital Universitario Clínico San Cecilio

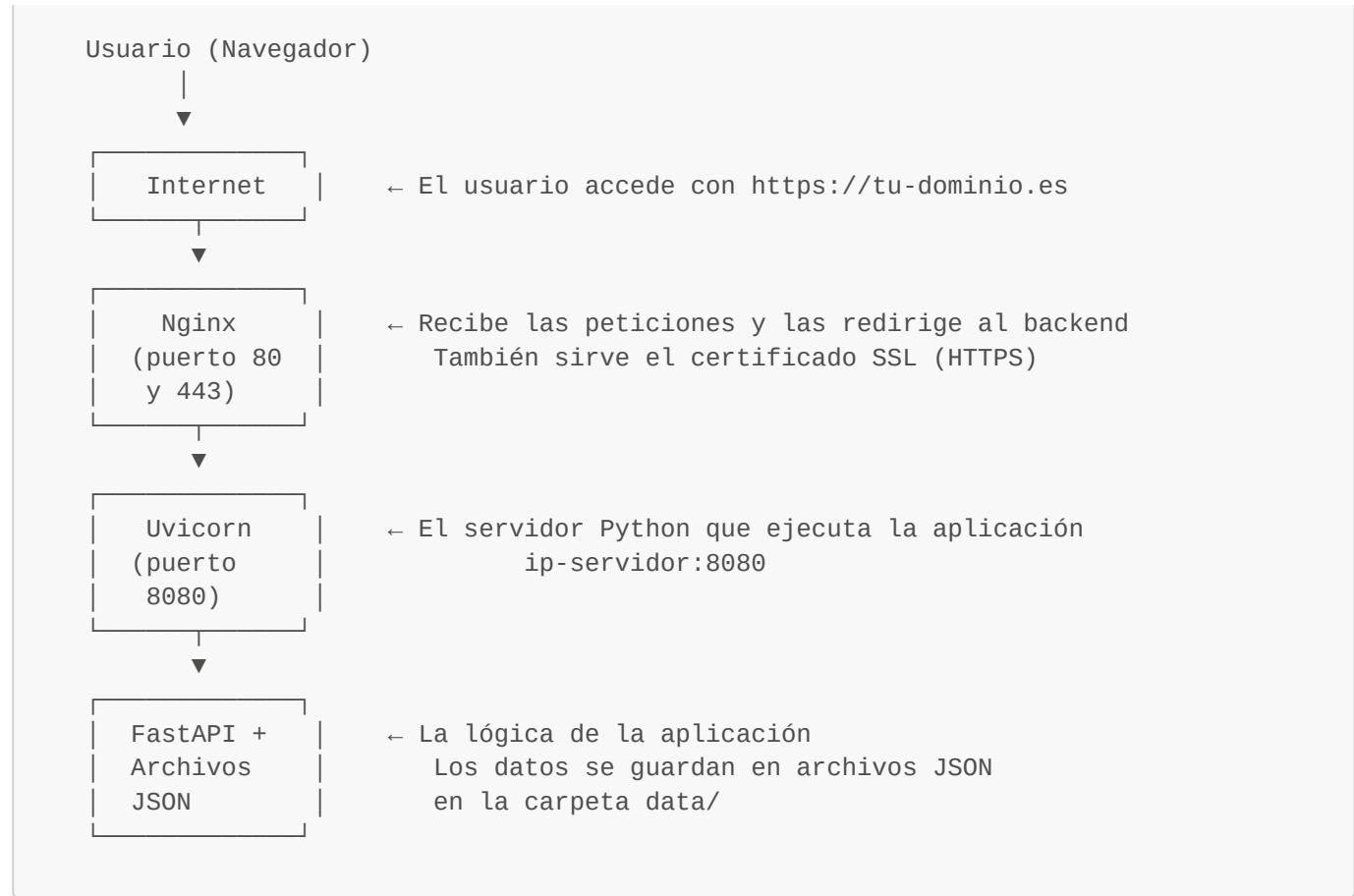
Guía paso a paso para desplegar la plataforma en un **servidor**, accesible desde internet.

Índice

- 5.5.1. [Resumen de la arquitectura](#)
- 5.5.2. [Requisitos del servidor](#)
- 5.5.3. [Preparar el servidor](#)
- 5.5.4. [Subir el proyecto al servidor](#)
- 5.5.5. [Instalar dependencias de Python](#)
- 5.5.6. [Configurar variables de entorno](#)
- 5.5.7. [Crear el servicio del sistema \(systemd\)](#)
- 5.5.8. [Configurar Nginx como proxy inverso](#)
- 5.5.9. [Obtener certificado SSL con Let's Encrypt](#)
- 5.5.10. [Configurar el firewall](#)
- 5.5.11. [Verificar el despliegue](#)
- 5.5.12. [Mantenimiento y copias de seguridad](#)
- 5.5.13. [Estructura del proyecto en el servidor](#)
- 5.5.14. [Solución de problemas](#)
- 5.5.15. [Resumen rápido \(Chuleta\)](#)

5.5.1. Resumen de la arquitectura

Así funciona la aplicación una vez desplegada en un servidor:



¿Por qué Nginx delante? Porque Nginx es un servidor web profesional que gestiona miles de conexiones simultáneas, maneja SSL/HTTPS, comprime respuestas y protege al backend de ataques directos.

5.5.2. Requisitos del servidor

Requisito	Mínimo recomendado
Sistema operativo	Preferiblemente Ubuntu 22.04 LTS / Debian 12 (esta guía usa Ubuntu) aunque admite cualquier distribución Linux o Windows Server
RAM	1 GB
Disco	10 GB
CPU	1 vCPU
Acceso	Conexión SSH al servidor
Dominio (opcional)	Un dominio apuntando a la IP del servidor (ej: <code>asociaciones.hospital.es</code>)
Puertos abiertos	22 (SSH), 80 (HTTP), 443 (HTTPS)

¿Dónde consigo un servidor? Si no se dispone de infraestructura interna, puedes usar proveedores como [Hetzner](#), [DigitalOcean](#), [OVH](#), o el propio departamento de IT del hospital si dispone de infraestructura interna.

5.5.3. Preparar el servidor

5.5.3.1. Conectarse al servidor por SSH

/

Desde tu ordenador Windows, abre **PowerShell** y conéctate:

```
ssh usuario@IP_DEL_SERVIDOR
```

Sustituye **usuario** por tu nombre de usuario del servidor e **IP_DEL_SERVIDOR** por la IP real (ej: `ssh admin@203.0.113.50`).

Si prefieres una interfaz gráfica, puedes usar **PutTY**: descárgalo, introduce la IP en "Host Name", puerto 22, y pulsa "Open".

5.5.3.2. Actualizar el sistema

```
sudo apt update && sudo apt upgrade -y
```

5.5.3.3. Instalar el software necesario

```
sudo apt install -y python3 python3-pip python3-venv nginx unzip ufw certbot  
python3-certbot-nginx
```

Paquete	¿Para qué sirve?
<code>python3</code> , <code>python3-pip</code> , <code>python3-venv</code>	El lenguaje del backend y su gestor de paquetes
<code>nginx</code>	Servidor web que actúa como proxy inverso
<code>unzip</code>	Para descomprimir el archivo del proyecto
<code>ufw</code>	Firewall sencillo para proteger el servidor
<code>certbot</code> + <code>python3-certbot-nginx</code>	Obtener certificados SSL gratuitos (Let's Encrypt)

5.5.3.4. Crear un usuario dedicado para la aplicación (buena práctica)

```
sudo useradd -m -s /bin/bash catalogo  
sudo passwd catalogo
```

Esto crea un usuario llamado **catalogo** con su propia carpeta. Se le pedirá establecer una contraseña.

5.5.4. Subir el proyecto al servidor

El proyecto se entrega como un archivo comprimido (**.zip**). Hay varias formas de subirlo al servidor desde Windows.

5.5.4.1. Opción A: Subir mediante WinSCP o FileZilla (recomendado)

Esta es la forma más sencilla desde Windows, ya que utiliza una interfaz gráfica:

1. Descarga e instala **WinSCP** o **FileZilla**.
2. Abre el programa y crea una nueva conexión:

- **Protocolo:** SFTP
- **Nombre del servidor:** la IP del servidor (ej: **203.0.113.50**)
- **Puerto:** 22
- **Usuario y Contraseña:** las credenciales de acceso al servidor

3. Pulsa **Conectar**.

4. En el panel derecho (servidor), navega a **/home/catalogo/**.

5. En el panel izquierdo (tu PC), busca la carpeta del proyecto.

6. Arrastra la carpeta del proyecto al panel derecho y renómbrala a **app**.

5.5.4.2. Opción B: Subir con SCP desde PowerShell

Abre **PowerShell** en Windows y ejecuta:

```
# Si el proyecto está en un archivo ZIP:
scp C:\Ruta\al\Catalogo_Asociaciones.zip usuario@IP_DEL_SERVIDOR:/home/catalogo/

# Si es una carpeta descomprimida:
scp -r C:\Ruta\al\Catalogo_Asociaciones\ usuario@IP_DEL_SERVIDOR:/home/catalogo/app
```

Si subiste un ZIP, conéctate al servidor y descomprímelo:

```
ssh usuario@IP_DEL_SERVIDOR
cd /home/catalogo
unzip Catalogo_Asociaciones.zip -d app
```

5.5.4.3. Opción C: Subir mediante un pendrive (servidores físicos)

Si el servidor es una máquina física accesible:

1. Copia el archivo comprimido a un pendrive USB.
2. Monta el pendrive en el servidor y copia los archivos a **/home/catalogo/app**.

5.5.4.4. Verificar que los archivos están correctamente subidos

```
ls /home/catalogo/app/
```

Deberías ver: **server.py**, **requirements.txt**, **html/**, **data/**, etc.

Asegúrate de que el usuario **catalogo** sea el propietario de todos los archivos:

```
sudo chown -R catalogo:catalogo /home/catalogo/app
```

5.5.5. Instalar dependencias de Python

```
# Cambiar al usuario de la aplicación
sudo -u catalogo -i
```

```
# Ir a la carpeta del proyecto
cd /home/catalogo/app

# Crear entorno virtual
python3 -m venv venv

# Activar entorno virtual
source venv/bin/activate

# Instalar dependencias
pip install -r requirements.txt

# Verificar que funciona (debería arrancar sin errores)
python -m uvicorn server:app --host 127.0.0.1 --port 8080 &
curl http://127.0.0.1:8080
# Si devuelve HTML, funciona correctamente

# Detener la prueba
kill %1
```

5.5.6. Configurar variables de entorno

Crea un archivo `.env` en la carpeta del proyecto:

```
sudo nano /home/catalogo/app/.env
```

Escribe lo siguiente (modifica los valores):

```
# =====
# CONFIGURACIÓN DE PRODUCCIÓN
# =====

# Clave secreta para firmar tokens JWT (CÁMBIALA por una cadena larga y aleatoria)
SECRET_KEY=pon_aquí_una_clave_muy_larga_y_aleatoria_2026_xyz

# Credenciales del administrador
ADMIN_USERNAME=admin
ADMIN_PASSWORD=ContraseñaDeAdministrador1!
```

Guarda con `Ctrl+O`, `Enter`, y cierra con `Ctrl+X`.

Protege el archivo para que solo el propietario pueda leerlo:

```
chmod 600 /home/catalogo/app/.env
chown catalogo:catalogo /home/catalogo/app/.env
```

IMPORTANTE: Nunca uses las credenciales por defecto (`admin` / `1234`) en el servidor

5.5.7. Crear el servicio del sistema (systemd)

Un **servicio systemd** hace que la aplicación se inicie automáticamente cuando el servidor arranca y se reinicie si se cae.

5.5.7.1. Crear el archivo de servicio

```
sudo nano /etc/systemd/system/catalogo.service
```

Pega este contenido:

```
[Unit]
Description=Catálogo de Asociaciones - AsociAcción
After=network.target

[Service]
Type=simple
User=catalogo
Group=catalogo
WorkingDirectory=/home/catalogo/app
EnvironmentFile=/home/catalogo/app/.env
ExecStart=/home/catalogo/app/venv/bin/uvicorn server:app --host 127.0.0.1 --port
8080 --workers 2
Restart=always
RestartSec=5

# Seguridad adicional del proceso
NoNewPrivileges=true
PrivateTmp=true

[Install]
WantedBy=multi-user.target
```

Guarda y cierra (**Ctrl+O**, **Enter**, **Ctrl+X**).

5.5.7.2. Activar y arrancar el servicio

```
# Recargar la configuración de systemd
sudo systemctl daemon-reload

# Activar para que arranque con el servidor
sudo systemctl enable catalogo

# Arrancar el servicio ahora
sudo systemctl start catalogo

# Verificar que está funcionando
sudo systemctl status catalogo
```

Deberías ver algo como:

- `catalogo.service` - Catálogo de Asociaciones - AsociAcción
Active: active (running) since ...

5.5.7.3. Comandos útiles del servicio

Acción	Comando
Ver estado	<code>sudo systemctl status catalogo</code>
Arrancar	<code>sudo systemctl start catalogo</code>
Detener	<code>sudo systemctl stop catalogo</code>
Reiniciar	<code>sudo systemctl restart catalogo</code>
Ver logs en tiempo real	<code>sudo journalctl -u catalogo -f</code>
Ver últimos 50 logs	<code>sudo journalctl -u catalogo -n 50</code>

5.5.8. Configurar Nginx como proxy inverso

Nginx recibe las peticiones de internet (puerto 80/443) y las redirige al backend (puerto 8080).

5.5.8.1. Crear la configuración del sitio

```
sudo nano /etc/nginx/sites-available/catalogo
```

Pega este contenido (sustituye `tu-dominio.es` por tu dominio real, o usa la IP del servidor):

```
server {
    listen 80;
    server_name tu-dominio.es www.tu-dominio.es;

    # Tamaño máximo de subida de archivos (logos, CSVs, fondos)
    client_max_body_size 15M;

    # Redirigir la raíz al index
    location = / {
        return 301 /html/index.html;
    }

    # Proxy inverso: redirigir todo al backend de FastAPI/Uvicorn
    location / {
        proxy_pass http://127.0.0.1:8080;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;

        # Timeouts generosos para importaciones grandes
        proxy_read_timeout 120s;
        proxy_send_timeout 120s;
    }
}
```

```
}  
}
```

5.5.8.2. Activar el sitio

```
# Crear enlace simbólico para activar el sitio  
sudo ln -s /etc/nginx/sites-available/catalogo /etc/nginx/sites-enabled/  
  
# Eliminar la página por defecto de Nginx (opcional)  
sudo rm -f /etc/nginx/sites-enabled/default  
  
# Verificar que la configuración no tiene errores  
sudo nginx -t  
  
# Recargar Nginx  
sudo systemctl reload nginx
```

Si `nginx -t` muestra `syntax is ok` y `test is successful`, todo está correcto.

5.5.8.3. Si no tienes dominio (solo IP)

Sustituye `tu-dominio.es` por `_` (acepta cualquier petición):

```
server_name _;
```

5.5.9. Obtener certificado SSL con Let's Encrypt

Requisito: Necesitas un dominio real apuntando a la IP del servidor. Si solo usas IP, salta este paso (usarás HTTP sin cifrar o un certificado autofirmado).

5.5.9.1. Generar el certificado (automático)

```
sudo certbot --nginx -d tu-dominio.es -d www.tu-dominio.es
```

Certbot te pedirá:

1. Un correo electrónico (para avisos de renovación).
2. Aceptar los términos de servicio.
3. Si quieres redirigir todo HTTP a HTTPS (recomendado: **Sí**).

5.5.9.2. Verificar la renovación automática

Let's Encrypt caduca cada 90 días, pero Certbot renueva automáticamente:

```
sudo certbot renew --dry-run
```

Si no muestra errores, la renovación automática está configurada.

5.5.9.3. Alternativa: Certificado autofirmado (sin dominio)

Si no tienes dominio, puedes usar el script incluido en el proyecto:

```
cd /home/catalogo/app
python3 generar_cert.py
```

Esto genera `cert.pem` y `key.pem`. Luego configura Nginx para usarlos:

```
server {
    listen 443 ssl;
    server_name _;

    ssl_certificate /home/catalogo/app/cert.pem;
    ssl_certificate_key /home/catalogo/app/key.pem;

    # ... resto de la configuración igual que antes
}
```

Con certificados autofirmados el navegador mostrará un aviso de seguridad.

5.5.10. Configurar el firewall

```
# Permitir SSH (para no perder acceso)
sudo ufw allow 22/tcp

# Permitir HTTP y HTTPS
sudo ufw allow 80/tcp
sudo ufw allow 443/tcp

# Activar el firewall
sudo ufw enable

# Verificar las reglas
sudo ufw status
```

Resultado esperado:

```
Status: active

To Action From
--
22/tcp ALLOW Anywhere
80/tcp ALLOW Anywhere
443/tcp ALLOW Anywhere
```


Nunca bloques el puerto 22 sin verificar que tienes otro método de acceso al servidor, o perderás la conexión SSH.

5.5.11. Verificar el despliegue

5.5.11.1. Comprobar que todo está corriendo

```
# 1. ¿El servicio está activo?
sudo systemctl status catalogo
# Debe mostrar: Active: active (running)

# 2. ¿Nginx está activo?
sudo systemctl status nginx
# Debe mostrar: Active: active (running)

# 3. ¿El backend responde localmente?
curl http://127.0.0.1:8080
# Debe devolver HTML

# 4. ¿Nginx responde desde fuera?
curl http://tu-dominio.es
# Debe devolver HTML (o redirigir a HTTPS)
```

5.5.11.2. Probar desde un navegador

Abre el navegador y accede a:

Página	URL
Catálogo público	https://tu-dominio.es
Login administrador	https://tu-dominio.es/html/login.html
Panel de gestión	https://tu-dominio.es/html/crearEntrada.html

5.5.12. Mantenimiento y copias de seguridad

5.5.12.1. Copias de seguridad automáticas

Crea un script de backup:

```
sudo nano /home/catalogo/backup.sh
```

```
#!/bin/bash
# Backup diario de los datos del Catálogo de Asociaciones
FECHA=$(date +%Y-%m-%d_%H%M)
BACKUP_DIR="/home/catalogo/backups"
mkdir -p "$BACKUP_DIR"

# Copiar la carpeta de datos y los logos
tar -czf "$BACKUP_DIR/backup_${FECHA}.tar.gz" \
```

```
/home/catalogo/app/data/ \
/home/catalogo/app/html/img/logos/ \
/home/catalogo/app/html/img/fondos/

# Eliminar backups de más de 30 días
find "$BACKUP_DIR" -name "backup_*.tar.gz" -mtime +30 -delete

echo "Backup completado: backup_${FECHA}.tar.gz"
```

Hazlo ejecutable y programa su ejecución diaria:

```
chmod +x /home/catalogo/backup.sh

# Añadir tarea programada (cron) – se ejecutará cada día a las 3:00 AM
sudo crontab -e
```

Añade esta línea al final:

```
0 3 * * * /home/catalogo/backup.sh >> /home/catalogo/backups/backup.log 2>&1
```

5.5.12.2. Actualizar la aplicación

Cuando se reciba una nueva versión del proyecto:

```
# 1. Detener el servicio
sudo systemctl stop catalogo

# 2. Hacer una copia de seguridad de los datos actuales
sudo cp -r /home/catalogo/app/data /home/catalogo/data_backup_$(date +%Y%m%d)
sudo cp -r /home/catalogo/app/html/img/logos /home/catalogo/logos_backup_$(date +%Y%m%d)

# 3. Subir la nueva versión al servidor (desde Windows con WinSCP o PowerShell):
#   Sube la nueva carpeta a /home/catalogo/app_nueva

# 4. En el servidor: reemplazar el código manteniendo los datos
sudo -u catalogo bash -c '
    cp -r /home/catalogo/app_nueva/* /home/catalogo/app/
    cd /home/catalogo/app
    source venv/bin/activate
    pip install -r requirements.txt
'

# 5. Restaurar los datos (si se sobrescribieron)
sudo cp -r /home/catalogo/data_backup_*//* /home/catalogo/app/data/

# 6. Reiniciar el servicio
sudo chown -R catalogo:catalogo /home/catalogo/app
sudo systemctl start catalogo
```

IMPORTANTE: Siempre haz una copia de seguridad de la carpeta `data/` y `html/img/logos/` antes de actualizar.

5.5.12.3. Ver los logs de la aplicación

```
# Logs en tiempo real (para detectar errores)
sudo journalctl -u catalogo -f

# Logs de Nginx (accesos y errores)
sudo tail -f /var/log/nginx/access.log
sudo tail -f /var/log/nginx/error.log
```

5.5.13. Estructura del proyecto en el servidor

```
/home/catalogo/
├── app/
│   ├── server.py
│   ├── generar_cert.py
│   ├── requirements.txt
│   ├── plantilla.csv
│   ├── .env
│   ├── venv/
│   └── html/
│       ├── index.html
│       ├── login.html
│       ├── crearEntrada.html
│       ├── contacto.html
│       ├── css/
│       ├── js/
│       └── img/
│           ├── logos/
│           └── fondos/
└── data/
    ├── asociaciones.json
    ├── categorias.json
    ├── etiquetas.json
    ├── config.json
    └── solicitudes.json

├── backups/
└── backup.sh

/etc/systemd/system/
└── catalogo.service

/etc/nginx/sites-available/
└── catalogo
```

- ← El proyecto completo
- ← Backend (servidor FastAPI)
- ← Generador de certificados SSL
- ← Dependencias Python
- ← Plantilla CSV de ejemplo
- ← Variables de entorno (credenciales)
- ← Entorno virtual de Python
- ← Frontend (páginas web)
- ← Página principal del catálogo
- ← Login del administrador
- ← Panel de gestión
- ← Formulario de solicitud
- ← Estilos visuales
- ← Lógica del frontend
- ← Imágenes
- ← Logos subidos de asociaciones
- ← Fondos de pantalla
- ← Base de datos (archivos JSON)
- ← Copias de seguridad automáticas
- ← Script de backup
- ← Servicio systemd
- ← Configuración de Nginx

5.5.14. Solución de problemas

5.5.14.1. "El servicio no arranca"

```
# Ver el error exacto
sudo journalctl -u catalogo -n 30 --no-pager
```

Causas habituales:

- **Ruta incorrecta** en `catalogo.service` → verifica que `/home/catalogo/app/venv/bin/uvicorn` existe.
- **Puerto ocupado** → otro proceso usa el 8080. Comprueba con `sudo lsof -i :8080`.
- **Error de Python** → activa el venv y ejecuta `python -m uvicorn server:app` manualmente para ver el error.

5.5.14.2. "Nginx muestra 502 Bad Gateway"

Significa que Nginx no puede conectar con el backend:

1. Verifica que el servicio `catalogo` está corriendo: `sudo systemctl status catalogo`
2. Verifica que el puerto coincide: el servicio usa `--port 8080` y Nginx apunta a `proxy_pass http://[IP_ADDRESS]:8080`.

5.5.14.3. "No puedo subir logos o archivos grandes"

Aumenta `client_max_body_size` en la configuración de Nginx:

```
client_max_body_size 25M;
```

Y recarga: `sudo systemctl reload nginx`

5.5.14.4. "Los datos se han perdido tras una actualización"

Los datos viven en `data/`. Si al actualizar se ha sobrescrito esa carpeta, restaura desde un backup:

```
cd /home/catalogo/backups
tar -xzf backup_2026-05-29_0300.tar.gz -C /
sudo systemctl restart catalogo
```

5.5.14.5. "El certificado SSL ha caducado"

```
sudo certbot renew
sudo systemctl reload nginx
```

5.5.14.6. "La sesión de admin caduca muy rápido"

Los tokens JWT expiran cada 30 minutos por seguridad. Esto es intencional. Solo hay que volver a iniciar sesión.

5.5.15. Resumen rápido (Chuleta)

Para administradores con experiencia, el despliegue completo resumido:

```
# 1. Preparar servidor
sudo apt update && sudo apt install -y python3 python3-pip python3-venv nginx unzip
ufw certbot python3-certbot-nginx

# 2. Crear usuario y subir proyecto
sudo useradd -m -s /bin/bash catalogo
# (Desde tu ordenador): scp Catalogo_Asociaciones.zip usuario@IP:/home/catalogo/
sudo -u catalogo unzip /home/catalogo/Catalogo_Asociaciones.zip -d
/home/catalogo/app
sudo chown -R catalogo:catalogo /home/catalogo/app

# 3. Instalar dependencias Python
sudo -u catalogo bash -c 'cd /home/catalogo/app && python3 -m venv venv && source
venv/bin/activate && pip install -r requirements.txt'

# 4. Crear .env con credenciales seguras
sudo -u catalogo bash -c 'cat > /home/catalogo/app/.env << EOF
SECRET_KEY=clave_super_secreta_aleatoria
ADMIN_USERNAME=admin
ADMIN_PASSWORD=ContraseñaDeAdministrador1!
EOF'
chmod 600 /home/catalogo/app/.env

# 5. Crear servicio systemd (ver sección 7)
# 6. Crear config Nginx (ver sección 8)

# 7. Activar todo
sudo systemctl daemon-reload
sudo systemctl enable --now catalogo
sudo ln -s /etc/nginx/sites-available/catalogo /etc/nginx/sites-enabled/
sudo nginx -t && sudo systemctl reload nginx

# 8. SSL + Firewall
sudo certbot --nginx -d tu-dominio.es
sudo ufw allow 22/tcp && sudo ufw allow 80/tcp && sudo ufw allow 443/tcp && sudo
ufw enable
```

5.6. Guía de Migración — De Python a Node.js y Base de Datos

Catálogo de Asociaciones — AsociAcción Hospital Universitario Clínico San Cecilio

Este documento plantea la hoja de ruta para una futura migración de la plataforma a un backend en **Node.js** con una **base de datos relacional (PostgreSQL)**, sustituyendo el actual backend en Python/FastAPI con almacenamiento en archivos JSON.

Índice

- 5.6.1. [¿Por qué migrar?](#)
- 5.6.2. [Arquitectura actual vs. arquitectura propuesta](#)
- 5.6.3. [Tecnologías propuestas](#)
- 5.6.4. [Plan de migración por fases](#)
- 5.6.5. [Fase 1 — Preparar la base de datos](#)
- 5.6.6. [Fase 2 — Crear el nuevo backend en Node.js](#)

- 5.6.7. [Fase 3 — Migrar los datos existentes](#)
- 5.6.8. [Fase 4 — Adaptar el frontend](#)
- 5.6.9. [Fase 5 — Pruebas y validación](#)
- 5.6.10. [Fase 6 — Despliegue y corte](#)
- 5.6.11. [Cambios en el despliegue del servidor](#)
- 5.6.12. [Riesgos y cómo mitigarlos](#)
- 5.6.13. [Estimación de esfuerzo](#)
- 5.6.14. [Decisiones pendientes](#)

5.6.1. ¿Por qué migrar?

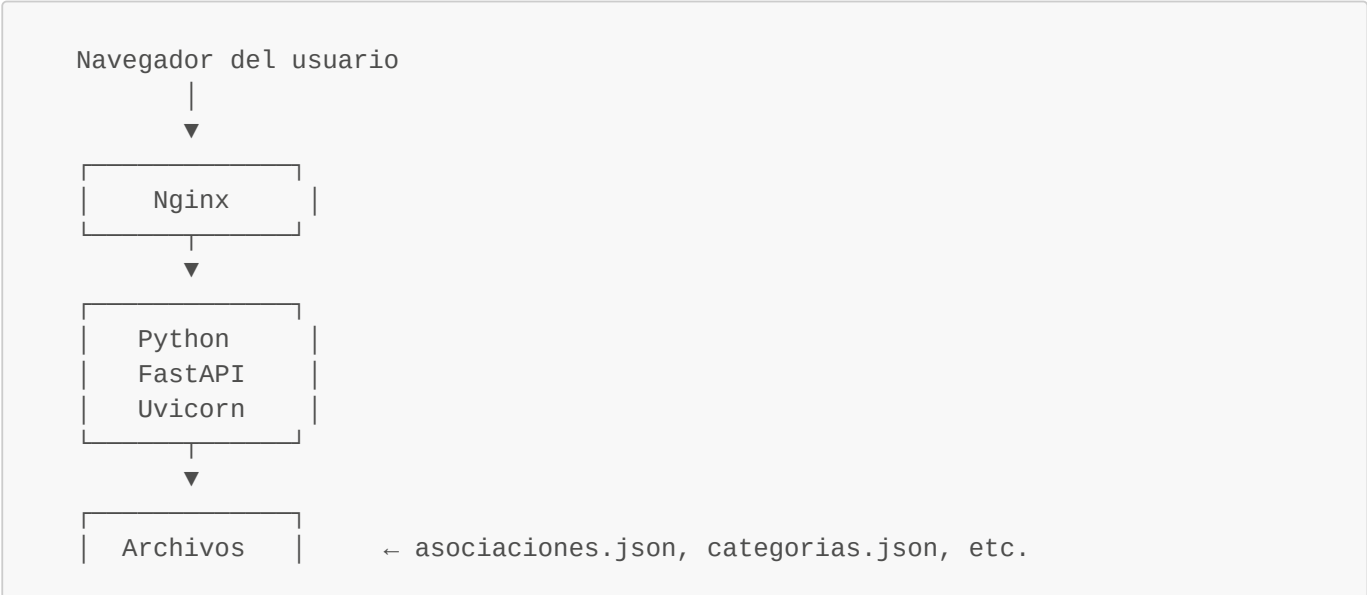
La aplicación actual funciona correctamente, pero tiene limitaciones que una migración resolvería:

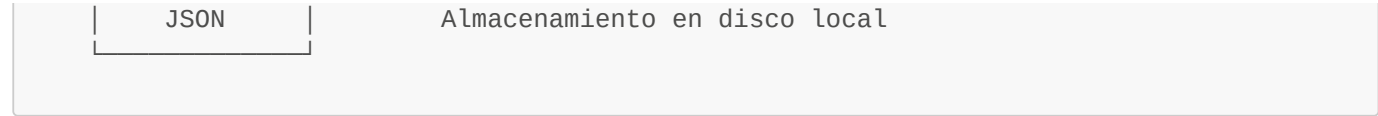
Limitación actual	Cómo lo resuelve la migración
Los datos se guardan en archivos JSON en disco	Una base de datos PostgreSQL ofrece consultas rápidas, integridad de datos y escalabilidad
Si dos usuarios modifican datos a la vez, uno puede sobrescribir al otro	PostgreSQL gestiona la conurrencia de forma segura con transacciones
No hay búsquedas avanzadas (ej: "asociaciones de Granada con más de 3 servicios")	SQL permite consultas complejas sobre los datos
Python requiere un entorno virtual y dependencias específicas	Node.js es ampliamente utilizado en entornos web y más fácil de desplegar en la nube
No hay copias de seguridad automáticas de los datos	PostgreSQL permite backups automáticos integrados
Si el servidor se reinicia en un entorno cloud, los archivos JSON pueden perderse	La base de datos es externa y persistente , los datos no se pierden

Nota importante: La migración no es urgente. El sistema actual es funcional y estable. Esta guía plantea un camino para cuando se necesite escalar o profesionalizar la plataforma.

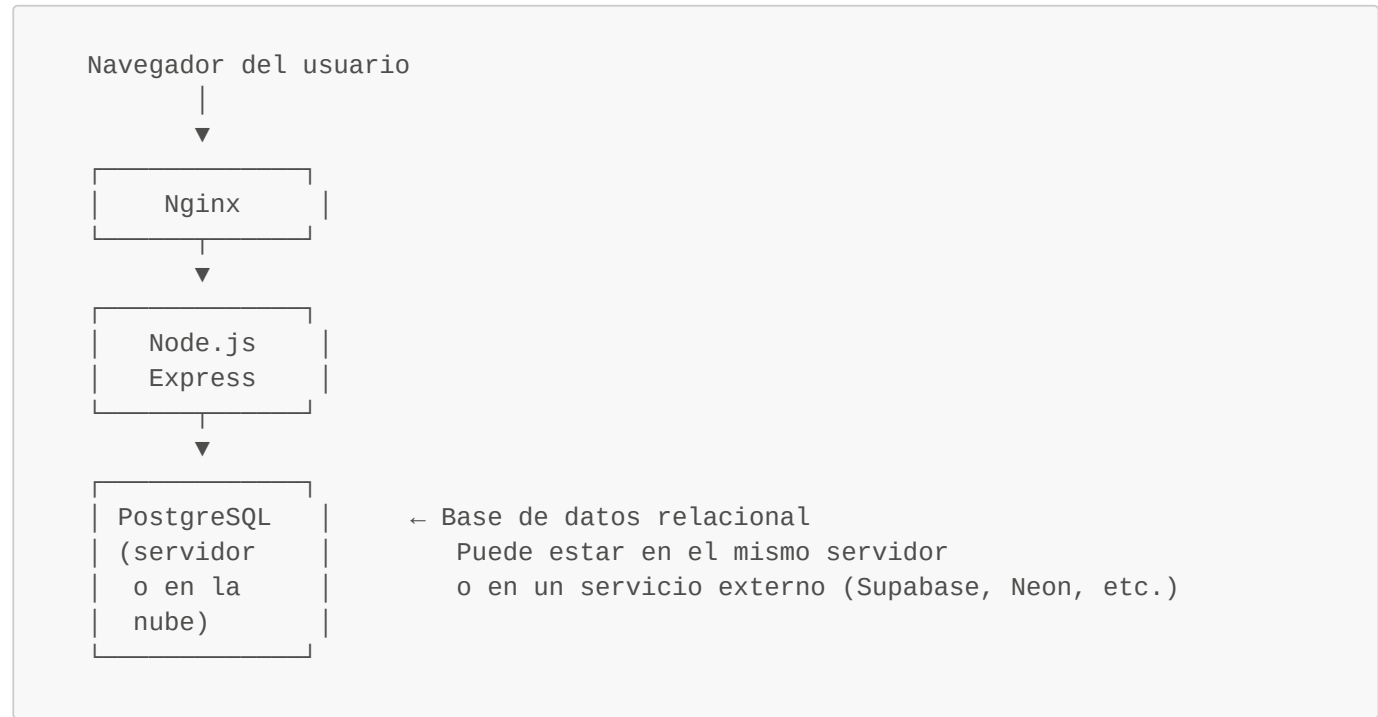
5.6.2. Arquitectura actual vs. arquitectura propuesta

5.6.2.1. Arquitectura actual





5.6.2.2. Arquitectura propuesta



5.6.2.3. ¿Qué cambia y qué no cambia?

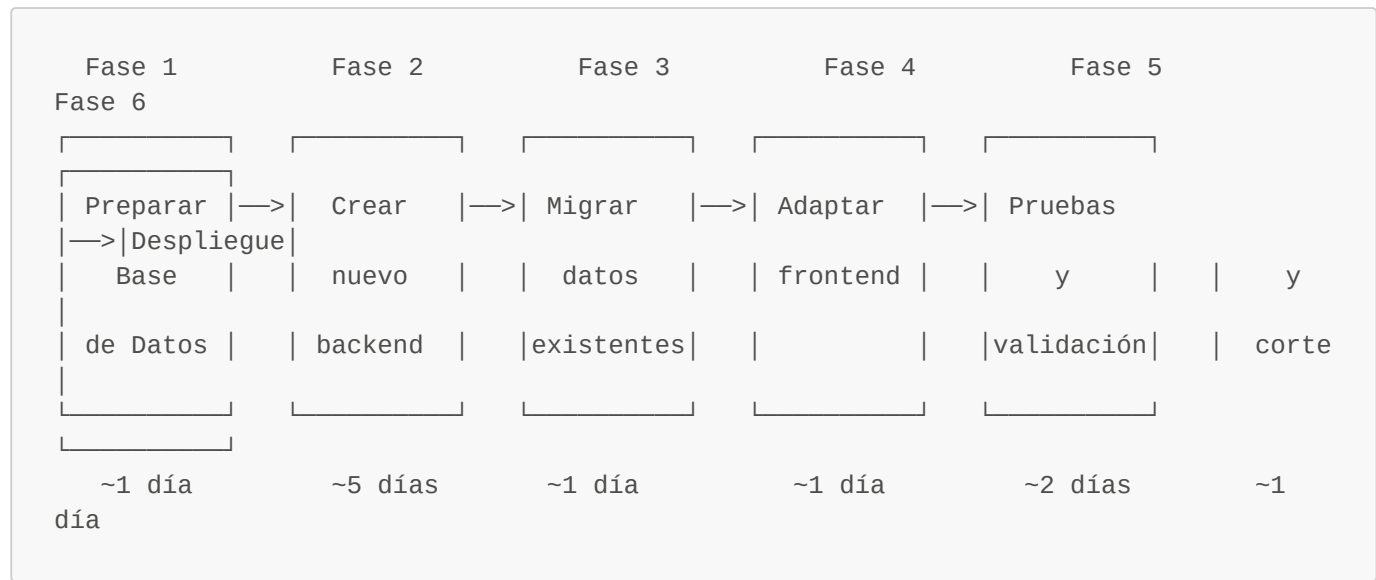
Componente	¿Cambia?	Detalle
Frontend (HTML, CSS, JS)	Apenas cambia	Solo se ajustan las URLs de la API si cambian
Backend (servidor)	Cambia por completo	De Python/FastAPI a Node.js/Express
Base de datos	Cambia por completo	De archivos JSON a tablas PostgreSQL
Nginx	No cambia	Sigue actuando como proxy inverso
Certificado SSL	No cambia	Let's Encrypt sigue funcionando igual
Firewall	No cambia	Mismas reglas

5.6.3. Tecnologías propuestas

Tecnología	Versión recomendada	Función
Node.js	20 LTS o superior	Entorno de ejecución del backend
Express	4.x	Framework web para la API
PostgreSQL	15 o superior	Base de datos relacional
pg (npm)	Última estable	Driver de Node.js para conectar con PostgreSQL
jsonwebtoken (npm)	Última estable	Gestión de tokens JWT (autenticación)
bcryptjs (npm)	Última estable	Hash seguro de contraseñas

Tecnología	Versión recomendada	Función
multer (npm)	Última estable	Gestión de subida de archivos
dotenv (npm)	Última estable	Lectura de variables de entorno (.env)
PM2	Última estable	Gestor de procesos (reemplaza a systemd para Node.js)

5.6.4. Plan de migración por fases



5.6.5. Fase 1 — Preparar la base de datos

5.6.5.1. Instalar PostgreSQL en el servidor

```
sudo apt install -y postgresql postgresql-contrib
sudo systemctl enable postgresql
sudo systemctl start postgresql
```

5.6.5.2. Crear la base de datos y el usuario

```
sudo -u postgres psql
```

```
-- Crear un usuario para la aplicación
CREATE USER catalogo_user WITH PASSWORD 'contraseña_segura';

-- Crear la base de datos
CREATE DATABASE catalogo_asociaciones OWNER catalogo_user;

-- Dar permisos
GRANT ALL PRIVILEGES ON DATABASE catalogo_asociaciones TO catalogo_user;

\q
```


5.6.5.3. Crear las tablas

Conectarse a la nueva base de datos:

```
sudo -u postgres psql -d catalogo_asociaciones
```

```
-- Tabla de categorías
CREATE TABLE categorias (
    id VARCHAR(50) PRIMARY KEY,
    nombre VARCHAR(255) NOT NULL
);

-- Tabla de etiquetas
CREATE TABLE etiquetas (
    id VARCHAR(50) PRIMARY KEY,
    nombre VARCHAR(255) NOT NULL
);

-- Tabla de asociaciones
CREATE TABLE asociaciones (
    id VARCHAR(50) PRIMARY KEY,
    nombre_asociacion VARCHAR(255) NOT NULL,
    siglas VARCHAR(50) NOT NULL,
    logo TEXT DEFAULT '',
    descripcion TEXT NOT NULL,
    categoria_id VARCHAR(50) REFERENCES categorias(id),
    cartera_servicios TEXT NOT NULL,
    videos JSONB DEFAULT '[]',
    ubicacion JSONB DEFAULT '{}',
    contactos JSONB DEFAULT '[]',
    etiquetas JSONB DEFAULT '[]'
);

-- Tabla de configuración del sitio
CREATE TABLE config (
    id SERIAL PRIMARY KEY,
    fondo TEXT DEFAULT ''
);

-- Tabla de solicitudes de inclusión
CREATE TABLE solicitudes (
    id VARCHAR(50) PRIMARY KEY,
    nombre_asociacion VARCHAR(255) NOT NULL,
    siglas VARCHAR(50) NOT NULL,
    descripcion TEXT NOT NULL,
    categoria_sugerida VARCHAR(255) DEFAULT '',
    web VARCHAR(500) DEFAULT '',
    ubicacion JSONB DEFAULT '{}',
    mensaje TEXT DEFAULT '',
    representante JSONB NOT NULL,
    fecha VARCHAR(50) NOT NULL,
    estado VARCHAR(20) DEFAULT 'pendiente'
);
```

5.6.5.4. Alternativa: usar PostgreSQL en la nube

En lugar de instalar PostgreSQL en el servidor, se puede usar un servicio gestionado (la base de datos se mantiene y respalda de forma automática):

Servicio	Plan gratuito	Ideal para
Supabase	Sí (500 MB)	Proyectos pequeños-medianos
Neon	Sí (512 MB)	Escalado automático
ElephantSQL	Sí (20 MB)	Pruebas y prototipos

En este caso, solo se necesita la **URL de conexión** que el servicio proporciona:

```
postgresql://catalogo_user:contraseña@servidor:5432/catalogo_asociaciones
```

5.6.6. Fase 2 — Crear el nuevo backend en Node.js

5.6.6.1. Instalar Node.js en el servidor

```
# Instalar Node.js 20 LTS
curl -fsSL https://deb.nodesource.com/setup_20.x | sudo -E bash -
sudo apt install -y nodejs

# Verificar
node --version    # Debe ser v20.x o superior
npm --version
```

5.6.6.2. Inicializar el proyecto

```
cd /home/catalogo/app
npm init -y
npm install express pg jsonwebtoken bcryptjs multer dotenv cors
```

5.6.6.3. Equivalencia entre endpoints actuales y nuevos

La API debe replicar exactamente los mismos endpoints para que el frontend no necesite cambios significativos:

Endpoint actual (Python)	Endpoint nuevo (Node.js)	Función
POST /token	POST /token	Login y obtención de token JWT
GET /api/asociaciones	GET /api/asociaciones	Listar asociaciones
POST /api/asociaciones	POST /api/asociaciones	Crear asociación
PUT /api/asociaciones/{id}	PUT /api/asociaciones/:id	Editar asociación
DELETE /api/asociaciones/{id}	DELETE /api/asociaciones/:id	Eliminar asociación

Endpoint actual (Python)	Endpoint nuevo (Node.js)	Función
GET /api/categorias	GET /api/categorias	Listar categorías
GET /api/etiquetas	GET /api/etiquetas	Listar etiquetas
GET /api/config	GET /api/config	Obtener configuración
POST /api/config	POST /api/config	Guardar configuración
POST /api/upload-logo	POST /api/upload-logo	Subir logo
GET /api/fondos	GET /api/fondos	Listar fondos
POST /api/upload-fondo	POST /api/upload-fondo	Subir fondo
POST /api/importar	POST /api/importar	Importar CSV/ODS
GET /api/exportar/csv	GET /api/exportar/csv	Exportar CSV
POST /api/solicitudes	POST /api/solicitudes	Enviar solicitud
GET /api/solicitudes	GET /api/solicitudes	Listar solicitudes

Regla clave: Si los endpoints y las respuestas JSON mantienen la misma estructura, el frontend no necesita cambios.

5.6.6.4. Estructura del nuevo backend

app/	
├─ server.js	← Punto de entrada principal
├─ package.json	← Dependencias de Node.js
├─ .env	← Variables de entorno
├─ db.js	← Conexión y consultas a PostgreSQL
├─ auth.js	← Lógica de autenticación JWT
├─ routes/	
│ ├─ asociaciones.js	← Endpoints de asociaciones
│ ├─ categorias.js	← Endpoints de categorías
│ ├─ etiquetas.js	← Endpoints de etiquetas
│ ├─ config.js	← Endpoints de configuración
│ ├─ solicitudes.js	← Endpoints de solicitudes
│ ├─ importar.js	← Lógica de importación
│ └─ exportar.js	← Lógica de exportación
├─ html/	← Frontend (sin cambios)
└─ data/	← Solo como respaldo local (opcional)

5.6.6.5. Variables de entorno (.env)

```
# Base de datos
DATABASE_URL=postgresql://catalogo_user:contraseña@localhost:5432/catalogo_asociaciones

# Autenticación
SECRET_KEY=clave_secreta_larga_y_aleatoria
ADMIN_USERNAME=admin
ADMIN_PASSWORD=ContraseñaDeAdministrador1!
```

```
# Servidor
PORT=8080
```

5.6.7. Fase 3 — Migrar los datos existentes

5.6.7.1. Script de migración de JSON a PostgreSQL

Se debe crear un script que lea los archivos JSON actuales y los inserte en la base de datos:

```
// migrar_datos.js – Ejecutar una sola vez
const { Pool } = require('pg');
const fs = require('fs');

const pool = new Pool({
  connectionString:
    'postgresql://catalogo_user:contraseña@localhost:5432/catalogo_asociaciones'
});

async function migrar() {
  // 1. Migrar categorías
  const categorias = JSON.parse(fs.readFileSync('data/categorias.json', 'utf8'));
  for (const cat of categorias) {
    await pool.query(
      'INSERT INTO categorias (id, nombre) VALUES ($1, $2) ON CONFLICT (id) DO NOTHING',
      [cat.id, cat.nombre]
    );
  }
  console.log(`Migradas ${categorias.length} categorías`);

  // 2. Migrar etiquetas
  const etiquetas = JSON.parse(fs.readFileSync('data/etiquetas.json', 'utf8'));
  for (const eti of etiquetas) {
    await pool.query(
      'INSERT INTO etiquetas (id, nombre) VALUES ($1, $2) ON CONFLICT (id) DO NOTHING',
      [eti.id, eti.nombre]
    );
  }
  console.log(`Migradas ${etiquetas.length} etiquetas`);

  // 3. Migrar asociaciones
  const asociaciones = JSON.parse(fs.readFileSync('data/asociaciones.json', 'utf8'));
  for (const aso of asociaciones) {
    await pool.query(
      `INSERT INTO asociaciones
      (id, nombre_asociacion, siglas, logo, descripcion, categoria_id,
      cartera_servicios, videos, ubicacion, contactos, etiquetas)
      VALUES ($1,$2,$3,$4,$5,$6,$7,$8,$9,$10,$11)
      ON CONFLICT (id) DO NOTHING`,
      [
        aso.id, aso.nombre_asociacion, aso.siglas, aso.logo || '',
        aso.descripcion, aso.categoria, aso.cartera_servicios,
        JSON.stringify(aso.videos || []),
      ]
    );
  }
}
```

```

        JSON.stringify(aso.ubicacion || {}),
        JSON.stringify(aso.contactos || []),
        JSON.stringify(aso.etiquetas || [])
    ]
    );
}
console.log(`Migradas ${asociaciones.length} asociaciones`);

// 4. Migrar solicitudes
if (fs.existsSync('data/solicitudes.json')) {
    const solicitudes = JSON.parse(fs.readFileSync('data/solicitudes.json',
'utf8'));
    for (const sol of solicitudes) {
        await pool.query(
            `INSERT INTO solicitudes
            (id, nombre_asociacion, siglas, descripcion, categoria_sugerida,
            web, ubicacion, mensaje, representante, fecha, estado)
            VALUES ($1,$2,$3,$4,$5,$6,$7,$8,$9,$10,$11)
            ON CONFLICT (id) DO NOTHING`,
            [
                sol.id, sol.nombre_asociacion, sol.siglas, sol.descripcion,
                sol.categoria_sugerida || '', sol.web || '',
                JSON.stringify(sol.ubicacion || {}), sol.mensaje || '',
                JSON.stringify(sol.representante), sol.fecha, sol.estado
            ]
        );
    }
    console.log(`Migradas ${solicitudes.length} solicitudes`);
}

console.log('\nMigración completada con éxito.');
```

```
pool.end();
}
```

```
migrar().catch(console.error);
```

5.6.7.2. Ejecutar la migración

```
cd /home/catalogo/app
node migrar_datos.js
```

5.6.7.3. Verificar los datos

```
sudo -u postgres psql -d catalogo_asociaciones -c "SELECT COUNT(*) FROM
asociaciones;"
sudo -u postgres psql -d catalogo_asociaciones -c "SELECT COUNT(*) FROM
categorias;"
sudo -u postgres psql -d catalogo_asociaciones -c "SELECT COUNT(*) FROM etiquetas;"
```

5.6.8. Fase 4 — Adaptar el frontend

El frontend (HTML, CSS, JavaScript) **no debería necesitar cambios significativos** si se respetan estas reglas:

- 1. **Los endpoints mantienen las mismas rutas** (/api/asociaciones, /api/categorias, etc.).
- 2. **Las respuestas JSON tienen la misma estructura** (mismos nombres de campos).
- 3. **El token de autenticación sigue siendo JWT** con el mismo formato.
- 4. **Los archivos estáticos se sirven desde la misma ruta** (/html/...).

5.6.8.1. Posibles ajustes menores

Caso	Qué revisar
El formato de <code>id</code> cambia (de número a texto o viceversa)	Ajustar comparaciones en JavaScript del frontend
La ubicación de los logos cambia	Actualizar las rutas en el frontend
El campo <code>categoria</code> pasa de almacenar un ID a almacenar un objeto	Adaptar la renderización en <code>index.js</code> y <code>detalle.js</code>

5.6.9. Fase 5 — Pruebas y validación

5.6.9.1. Lista de comprobación funcional

Función	Verificación
Página principal carga	Las asociaciones se muestran correctamente
Filtros funcionan	Categoría, etiqueta, provincia, comunidad, país
Detalle de asociación	Muestra contactos, servicios, vídeos
Login de administrador	Devuelve token y redirige al panel
Crear asociación	Se guarda en la base de datos
Editar asociación	Se actualiza correctamente
Eliminar asociación	Desaparece del listado
Subir logo	Se almacena y se muestra en la ficha
Importar CSV	Se procesan las filas correctamente
Exportar CSV	Se descarga con todos los datos
Formulario de solicitud	Se envía y aparece en el panel de admin
Cambiar fondo	Se aplica en la página principal

5.6.9.2. Pruebas de rendimiento

Comparar tiempos de respuesta entre la versión actual (JSON) y la nueva (PostgreSQL):

```
# Medir tiempo de respuesta del listado de asociaciones
time curl -s http://localhost:8080/api/asociaciones > /dev/null
```

5.6.10. Fase 6 — Despliegue y corte

5.6.10.1. Estrategia de corte recomendada

1. **Instalar la nueva versión** en una carpeta separada (/home/catalogo/app_nueva).
2. **Ejecutar el script de migración** de datos.
3. **Probar la nueva versión** en un puerto diferente (ej: 8081).
4. **Cuando todo funcione**, detener el servicio antiguo y redirigir Nginx al nuevo.
5. **Conservar la versión anterior** durante al menos una semana por si hay que volver atrás.

5.6.10.2. Cambiar Nginx al nuevo backend

Solo hay que modificar el puerto en la configuración de Nginx:

```
proxy_pass http://127.0.0.1:8080;    # Apuntar al nuevo backend Node.js
```

```
sudo nginx -t && sudo systemctl reload nginx
```

5.6.11. Cambios en el despliegue del servidor

5.6.11.1. Diferencias respecto al despliegue actual

Aspecto	Versión actual (Python)	Versión nueva (Node.js)
Lenguaje	Python 3.9+	Node.js 20+
Framework	FastAPI + Uvicorn	Express
Gestor de paquetes	pip + requirements.txt	npm + package.json
Entorno virtual	python3 -m venv venv	No necesario (usa node_modules/)
Instalar dependencias	pip install -r requirements.txt	npm install
Arrancar	uvicorn server:app --port 8080	node server.js o pm2 start server.js
Gestor de procesos	systemd	PM2 (recomendado) o systemd
Base de datos	Archivos JSON en data/	PostgreSQL (local o en la nube)
Backups	Copiar carpeta data/	pg_dump (volcado de base de datos)

5.6.11.2. Nuevo servicio systemd (si no se usa PM2)

```
[Unit]
Description=Catálogo de Asociaciones - Node.js
After=network.target postgresql.service

[Service]
Type=simple
```

```
User=catalogo
Group=catalogo
WorkingDirectory=/home/catalogo/app
EnvironmentFile=/home/catalogo/app/.env
ExecStart=/usr/bin/node server.js
Restart=always
RestartSec=5

[Install]
WantedBy=multi-user.target
```

5.6.11.3. Backups de PostgreSQL

En lugar de copiar archivos JSON, los backups se hacen con `pg_dump`:

```
# Backup manual
pg_dump -U catalogo_user catalogo_asociaciones > backup_$(date +%Y%m%d).sql

# Restaurar
psql -U catalogo_user catalogo_asociaciones < backup_20260529.sql
```

Script de backup automático (reemplaza al actual):

```
#!/bin/bash
FECHA=$(date +%Y-%m-%d_%H%M)
BACKUP_DIR="/home/catalogo/backups"
mkdir -p "$BACKUP_DIR"

# Backup de la base de datos
pg_dump -U catalogo_user catalogo_asociaciones | gzip >
"$BACKUP_DIR/db_${FECHA}.sql.gz"

# Backup de los logos y fondos (siguen en disco)
tar -czf "$BACKUP_DIR/archivos_${FECHA}.tar.gz" \
/home/catalogo/app/html/img/logos/ \
/home/catalogo/app/html/img/fondos/

# Limpiar backups antiguos (más de 30 días)
find "$BACKUP_DIR" -name "*.gz" -mtime +30 -delete

echo "Backup completado: ${FECHA}"
```

5.6.12. Riesgos y cómo mitigarlos

Riesgo	Probabilidad	Impacto	Cómo mitigarlo
Pérdida de datos durante la migración	Baja	Alto	Hacer backup completo antes de migrar. Conservar los JSON originales
El frontend deja de funcionar	Media	Alto	Mantener la misma estructura de endpoints y respuestas JSON

Riesgo	Probabilidad	Impacto	Cómo mitigarlo
PostgreSQL se cae	Baja	Alto	Implementar un sistema de fallback a archivos JSON locales como respaldo
Errores de traducción Python → JavaScript	Media	Medio	Buscar restos de sintaxis Python (<code>append</code> , <code>None</code> , <code>True</code>) en el código JS
El equipo no tiene experiencia en Node.js	Media	Medio	Documentar bien y usar código sencillo. Node.js tiene una comunidad muy grande
Tiempo de inactividad durante el corte	Baja	Bajo	Desplegar la nueva versión en paralelo y hacer el cambio en Nginx en segundos

5.6.13. Estimación de esfuerzo

Fase	Duración estimada	Perfil necesario
Fase 1: Base de datos	1 día	Técnico con conocimientos de SQL
Fase 2: Backend Node.js	3–5 días	Desarrollador con experiencia en JavaScript/Node.js
Fase 3: Migración de datos	0,5–1 día	Técnico
Fase 4: Ajustes frontend	0,5–1 día	Desarrollador frontend
Fase 5: Pruebas	1–2 días	QA o el propio desarrollador
Fase 6: Despliegue	0,5 día	Administrador de sistemas
Total estimado	7–10 días laborables	

5.6.14. Decisiones pendientes

Antes de iniciar la migración, se deben tomar las siguientes decisiones:

Decisión	Opciones	Recomendación
¿PostgreSQL local o en la nube?	Local (en el servidor) / Nube (Supabase, Neon)	Nube si se quiere reducir mantenimiento; local si se prefiere control total
¿Usar PM2 o systemd?	PM2 (más funcionalidades para Node.js) / systemd (ya configurado)	PM2 si el equipo lo conoce; systemd si se quiere consistencia con la infraestructura actual
¿Migrar las contraseñas a hash seguro?	Mantener texto plano como ahora / Migrar a bcrypt	Migrar a bcrypt para mayor seguridad
¿Mantener el fallback a JSON?	Sí (como red de seguridad) / No (solo base de datos)	Sí durante los primeros meses; eliminar cuando sea estable
¿Cuándo se ejecuta la migración?	Coordinarlo con un periodo de baja actividad del portal	Elegir un viernes por la tarde o un fin de semana

6. PRUEBAS Y VALIDACIÓN

6.1. Introducción

Este capítulo detalla la batería de pruebas diseñada y ejecutada tras concluir la fase de implementación. El objetivo de estas pruebas es verificar que el portal web cumple con los requerimientos funcionales, arquitectónicos y de seguridad definidos en la planificación del proyecto.

Las pruebas se han dividido en tres categorías. Las pruebas funcionales se encargan de validar la operatividad de los flujos de interacción para usuarios y administradores. Las pruebas de seguridad evalúan la protección del sistema ante accesos no autorizados y vulnerabilidades web comunes. Por último, las pruebas de compatibilidad certifican el correcto funcionamiento del diseño responsivo en diversos entornos de renderizado y navegadores.

6.2. Pruebas Funcionales

Las pruebas funcionales se enfocan en confirmar que cada módulo de la aplicación responde adecuadamente a las acciones esperadas.

Para el usuario no registrado, se evaluó el renderizado inicial del catálogo. Al acceder a la raíz del directorio, el sistema lee correctamente el archivo JSON y renderiza la cuadrícula de tarjetas de forma inmediata, inyectando imágenes por defecto en aquellos registros sin logotipo asignado. Posteriormente, se comprobó el filtrado dinámico mediante la selección concurrente de categorías y etiquetas en los menús desplegables; la interfaz actualiza los resultados en el cliente de manera instantánea filtrando los nodos sin generar peticiones adicionales al servidor. Finalmente, la prueba del buscador de texto libre demostró una respuesta reactiva, filtrando las coincidencias por siglas y nombres mientras se teclea, sin distinguir entre mayúsculas y minúsculas.

Para el usuario administrador, se validó el mecanismo de creación de entradas. Tras rellenar un formulario con todos los datos requeridos y adjuntar una imagen válida, el backend recibe la petición, valida la estructura mediante Pydantic, renombra la imagen con un identificador único, persiste los datos y devuelve un código de éxito que actualiza la interfaz. También se ejecutaron pruebas de manejo de errores, como enviar un formulario omitiendo campos obligatorios; en este caso, FastAPI intercepta la petición inválida, rechaza la escritura y devuelve un error HTTP 422, preservando la integridad del archivo de datos. Por último, se comprobó que las opciones de modificación y borrado operan de forma precisa, actualizando datos descriptivos sin alterar identificadores, o eliminando entidades completas de la base de datos y de la vista en tiempo real.

Adicionalmente, se auditó exhaustivamente el **Módulo de Gestión de Categorías**:

- **Consistencia de Cuentas:** Se verificó que cada tarjeta reflejase con precisión la cantidad de asociaciones asignadas cruzando los datos devueltos por `/api/asociaciones`.
 - **Integridad Referencial en Borrado:** Al pulsar el botón de eliminación en categorías con vinculaciones activas, se comprobó que el botón estuviera deshabilitado nativamente y que el tooltip visual advirtiera de la imposibilidad de la acción. Para categorías con 0 asociaciones, el borrado se ejecutó correctamente en el servidor mediante `DELETE` tras confirmación.
 - **Sincronización en Caliente:** Al confirmarse la supresión de una categoría vacía, se verificó la desaparición inmediata de su tarjeta y su remoción en el selector dinámico del formulario de entrada de datos sin provocar recargas del navegador.
-

6.3. Pruebas de Seguridad y Control de Accesos

Se realizaron pruebas específicas para verificar la protección del sistema ante accesos ilegítimos y vulnerabilidades comunes, dado el carácter sensible de la administración de la plataforma.

En el apartado de autenticación JWT, se simuló un intento de acceso directo a los endpoints protegidos (API) sin incluir la cabecera de autorización. El middleware de FastAPI rechazó correctamente la conexión devolviendo un estado HTTP

401. Asimismo, se auditó la expiración del token iniciando una sesión legítima y rebasando el límite temporal de caducidad. Al intentar realizar una modificación posterior, el servidor detectó la caducidad temporal de la firma, devolvió un error de acceso no autorizado y el script cliente forzó la redirección al formulario de autenticación, limpiando las variables locales.

Para prevenir la inyección de código (*Cross-Site Scripting*), se insertaron cadenas maliciosas de JavaScript en los campos de los formularios. La interfaz neutraliza estos vectores de ataque gracias a la manipulación segura del DOM mediante el uso exclusivo de la propiedad `textContent`, que renderiza el contenido como texto plano. Además, las funciones de sanitización bloquearon el uso de pseudoprotocolos maliciosos (como `javascript:`) en las direcciones web. Adicionalmente, el formulario de contacto público se evaluó mediante herramientas que simulan tráfico bot, comprobando que el campo oculto (Honeypot) intercepta con éxito envíos automatizados, abortando la petición HTTP localmente y protegiendo el backend de sobrecargas de spam.

6.4. Pruebas de Compatibilidad y Diseño (Frontend)

Para garantizar una experiencia de usuario uniforme en múltiples plataformas, se ejecutaron pruebas de renderizado visual.

La adaptabilidad a dispositivos móviles se confirmó redimensionando la ventana del navegador. Las reglas de maquetación fundamentadas en CSS Grid reestructuraron correctamente la cuadrícula principal, pasando de varias columnas a un diseño de columna única en resoluciones correspondientes a teléfonos móviles. El menú de navegación se ocultó adecuadamente, sustituyéndose por elementos desplegables que previenen el desbordamiento horizontal, mientras que los formularios ajustaron su ancho para ocupar la totalidad de la pantalla.

Se realizaron pruebas de robustez visual ante sobrecarga de información:

- **Prueba de Títulos Extensos:** Se inyectaron nombres de asociaciones de más de 80 caracteres. El script cliente truncó con éxito la denominación a 55 caracteres en la tarjeta pública añadiendo los puntos suspensivos e inyectando el tooltip nativo `title` visible en hover.
- **Prueba de Saturación de Etiquetas:** Se asignaron más de 12 etiquetas secundarias a un solo registro. Las reglas de maquetación CSS contuvieron las etiquetas en un máximo de dos filas (`max-height: 3.6rem` con envoltura), manteniendo perfectamente alineado el botón de detalles de la tarjeta y el resto de la rejilla.
- **Páginas Institucionales y Fondo Dinámico:** Se visitaron las nuevas páginas enlazadas del pie de página (`guia.html`, `aviso.html`, `privacidad.html`, `accesibilidad.html`) comprobando que cargaran correctamente el fondo de pantalla personalizado establecido desde el panel de administración HUSC de forma síncrona.

La prueba de compatibilidad cruzada de navegadores consistió en acceder al portal desde las versiones más recientes de Google Chrome, Mozilla Firefox, Microsoft Edge y Safari. Las validaciones arrojaron resultados satisfactorios: las propiedades CSS avanzadas, las transiciones y las tipografías corporativas se renderizaron correctamente y sin discrepancias en todos los motores probados.

6.5. Conclusión de las Pruebas

Los resultados del ciclo de auditoría técnica certifican la estabilidad y seguridad de la aplicación web. Los mecanismos de protección JWT y las defensas contra XSS operan correctamente, mitigando las amenazas perimetrales evaluadas. A nivel funcional, la lógica cliente desarrollada en JavaScript procesa los datos JSON del servidor asíncrono con precisión y rendimiento adecuado. Por lo tanto, el software cumple los estándares requeridos y se declara apto para su paso a entorno de producción o despliegue definitivo.

7. CONCLUSIÓN Y RESULTADOS

7.1. Introducción

En este último capítulo se realiza una evaluación global del proyecto desarrollado para el Hospital Universitario Clínico San Cecilio. El objetivo es analizar el cumplimiento de los requerimientos iniciales, evaluar los resultados técnicos obtenidos y proponer líneas de mejora funcional y arquitectónica para futuras iteraciones de la plataforma.

7.2. Consecución de los Objetivos

Tras la finalización de las fases de diseño, implementación y pruebas, los objetivos definidos en el estudio de viabilidad se han alcanzado satisfactoriamente.

Se ha logrado centralizar la información, agrupando los datos de las distintas asociaciones médicas y de pacientes vinculadas al área de influencia del hospital en un único directorio digital accesible. Frente a catálogos estáticos, la plataforma implementada ofrece un sistema funcional de categorización clínica, etiquetado múltiple y un motor de búsqueda asíncrono que facilita la localización rápida de recursos de apoyo por parte de los usuarios.

Para la gestión de la plataforma, se ha desarrollado un panel de administración que otorga autonomía a la Comisión de Participación Ciudadana. Este portal privado permite a los usuarios autorizados crear, editar, subir logotipos y eliminar entradas de manera intuitiva y visual, eliminando la dependencia de perfiles técnicos para la actualización periódica del contenido. Además, el proyecto se ha construido bajo un enfoque de desarrollo seguro, utilizando FastAPI para mitigar vulnerabilidades web, implementando autenticación mediante tokens JWT y añadiendo mecanismos preventivos antispam y validación estricta de esquemas.

7.3. Resultados Obtenidos y Beneficios

La implantación de este proyecto aporta beneficios directos tanto a los ciudadanos como a la administración del hospital.

Desde el punto de vista del usuario no registrado o paciente, la plataforma mejora la accesibilidad a la información. El diseño responsivo garantiza una navegación fluida desde dispositivos móviles y de escritorio. La presentación estructurada de datos de contacto, carteras de servicios, vídeos institucionales y ubicaciones reduce la curva de aprendizaje, permitiendo que cualquier visitante encuentre el apoyo requerido de forma clara y directa.

Para el perfil de administrador, la eficiencia operativa se incrementa significativamente. La publicación de nuevas asociaciones se realiza en pocos pasos gracias al sistema de gestión y carga de imágenes optimizado. Asimismo, la integración del formulario público automatiza la recepción de solicitudes de inclusión externas; estas peticiones se estandarizan, se limpian de envíos ilegítimos mediante barreras antispam, y se centralizan ordenadamente en la bandeja de auditoría para su revisión.

7.4. Líneas de Mejora del Proyecto (Trabajo Futuro)

A pesar de que el producto final cumple con los requerimientos técnicos fijados, se plantean diversas ampliaciones y mejoras arquitectónicas para asegurar la escalabilidad del sistema a medio y largo plazo.

En relación a la persistencia de datos, el sistema actual opera mediante archivos JSON. Ante un futuro incremento significativo del volumen de asociaciones y peticiones concurrentes, sería recomendable migrar la capa de persistencia a un motor de base de datos relacional robusto (como MySQL o PostgreSQL), implementando un modelo entidad-relación formal que mejore la transaccionalidad.

A nivel de seguridad y roles, la autenticación actual se basa en credenciales estandarizadas en variables de entorno. Una línea de mejora consistiría en el diseño de un sistema de gestión de usuarios dinámico, con almacenamiento de

contraseñas cifradas y establecimiento de privilegios granulares o roles jerárquicos (por ejemplo, administradores y superadministradores), lo que permitiría mantener registros de auditoría por usuario.

Para optimizar el rendimiento de la red y el renderizado en cliente, en caso de que el censo de entidades crezca de forma notable, será necesario implementar un sistema de paginación en el servidor. Modificando los endpoints para admitir parámetros de paginación, el backend podría enviar paquetes fraccionados de datos bajo demanda, aliviando el ancho de banda y el consumo de recursos en el navegador.

Finalmente, se propone incorporar un módulo de estadísticas en el panel de administración que registre métricas de uso y búsquedas frecuentes, aportando valor analítico a la toma de decisiones institucionales. Del mismo modo, la plataforma podría ampliarse con un sistema de tablón de noticias o agenda de eventos, permitiendo a las asociaciones comunicar talleres o reuniones de manera directa y dinámica dentro de la web del hospital.

8. BIBLIOGRAFÍA Y REFERENCIAS

En este apartado se detallan todas las fuentes de información, documentación oficial de lenguajes y librerías, así como los estándares web y la bibliografía técnica exhaustiva consultada para el diseño, el desarrollo estructurado y la implementación segura del portal web del Hospital Universitario Clínico San Cecilio.

8.1. Documentación Oficial de Lenguajes y Entornos

El desarrollo estructural del frontend se apoyó intensivamente en la documentación de la Mozilla Developer Network (MDN Web Docs), sirviendo como la guía principal para la implementación de la capa de presentación mediante HTML5 semántico, maquetación avanzada con CSS3 (incluyendo directivas de Flexbox y CSS Grid) y el uso nativo de la API de JavaScript puro para manipular el DOM y gestionar las peticiones asíncronas con Fetch API.

En lo que respecta al servidor, se recurrió constantemente a la documentación oficial de Python 3 para asegurar un manejo nativo y optimizado de las operaciones de lectura y escritura de archivos físicos mediante el módulo JSON, así como para la gestión segura de directorios del sistema operativo y la captura robusta de excepciones genéricas. Paralelamente, la construcción del backend dependió de la documentación oficial del framework FastAPI, la cual fue consultada en profundidad para diseñar el enrutamiento inteligente, configurar la validación automatizada de esquemas y estructurar el manejo de middlewares interceptores. Todo este andamiaje lógico fue ejecutado apoyándose en la documentación de Uvicorn, el servidor de interfaz de pasarela asíncrona elegido para soportar la ejecución concurrente de la aplicación web en Python.

8.2. Librerías y Dependencias del Proyecto

La inmutabilidad y validación de los datos descansó sobre la librería Pydantic. Su extensa documentación permitió dominar la gestión de configuraciones y la validación estricta utilizando anotaciones de tipos de Python, siendo un pilar esencial para depurar los esquemas correspondientes a asociaciones, ubicaciones y contactos antes de permitir su persistencia en el disco.

Para garantizar el control de acceso, se empleó la librería PyJWT, cuya referencia fue crucial para implementar la codificación y decodificación de tokens bajo el estricto estándar de la industria, garantizando así la generación de firmas criptográficas irrompibles durante el proceso de inicio de sesión de los administradores. Asimismo, la interceptación y lectura de archivos en streaming, como los logotipos de las asociaciones, fue posible gracias a la consulta del analizador python-multipart, un decodificador requerido intrínsecamente por FastAPI para procesar datos en formato de formulario codificado.

8.3. Estándares y Protocolos Web

La arquitectura de seguridad se basó fundamentalmente en la especificación abierta JSON Web Token (RFC 7519) dictada por la IETF, la cual definió el modo compacto e inviolable para transmitir información de identidad entre el

cliente y el servidor. La comprensión práctica de este estándar se consolidó a través de plataformas interactivas de depuración proporcionadas por Auth0.

Por otro lado, todas las defensas y mitigaciones de ataques inyectados se diseñaron tomando como referencia innegable las directrices del Open Web Application Security Project (OWASP). Sus documentos sobre ciberseguridad aplicada dictaron las normas para prevenir ataques de Cross-Site Scripting (XSS) obligando al uso de texto plano en el cliente, y guiaron el endurecimiento extremo del backend mediante la inyección meticulosa de políticas de seguridad de contenido (CSP) en las cabeceras HTTP. Este blindaje se completó siguiendo las normativas del estándar de Intercambio de Recursos de Origen Cruzado (CORS) consultadas directamente en MDN y aplicadas a través del middleware de FastAPI.

8.4. Recursos Gráficos y Herramientas

A nivel estético, la interfaz de usuario adquirió su identidad visual corporativa y moderna gracias al servicio de fuentes en la nube de Google Fonts, desde donde se importaron y sirvieron tipografías optimizadas para lectura digital. Finalmente, la materialización del código en todas sus capas, al igual que la redacción exhaustiva de este mismo documento técnico en formato Markdown, se llevó a cabo utilizando Visual Studio Code, un entorno de desarrollo integrado que centralizó y agilizó toda la operativa del proyecto.